Project Report Template

Advanced Analytics and Applications

Team 07

Authors:
Larissa Theilmann (7432976)
Emre Uyar (7443638)
Tjorge Orlitz (7443662)
Nils Hornstein (7369566)
Contact: torlitz@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter
Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-07-26

Table of contents

List of Figures

1 Introduction

1.1 Motivation and Problem Statement

The ride-hailing market is already crowded. Uber, Lyft, FreeNow, Lime, and Tier compete for the same trips, and current market reports point to ride-hailing, increasingly paired with autonomous and electric vehicles, as the segment that will dominate urban mobility in the near term. Our client, a well-established German car manufacturer, is building a ride-hailing platform on a fully electrified vehicle fleet, wanting to launch first in the United States, where customers have proven more receptive to novel mobility business models.

The client can design and build vehicles, but has no experience running a ride-hailing fleet. Managing an electric fleet adds a further constraint: Charging schedules directly affect vehicle availability in ways a conventional fleet does not face. To plan routes, size a fleet, and decide where to deploy vehicles, the client needs a clear picture of how demand moves through a city across space and time.

This report builds from historical taxi trip data for Chicago, made available by the Chicago Data Portal and covering trips from 2024 onward. Taxis are not ride-hailing vehicles, but in a market the client has not yet entered, they are the closest available proxy for one: metered urban trips, dispatched on demand, subject to the same weather and time-of-day patterns that will govern the client's own fleet. We supplement the trip records with independently collected weather data and, where it adds value, point-of-interest data from OpenStreetMap, and we analyze demand at spatial resolutions ranging from coarse census tracts to fine-grained H3 hexagon grids, and at multiple temporal resolutions, to establish which level of detail the data can actually support. We also investigate how a vehicle should charge at a driver's home between shifts, given a battery of finite capacity and a next-shift energy requirement that is not known with certainty in advance, using Reinforcement Learning.

1.2 Research Questions

The analysis that follows is organized around five questions, each feeding into the next.

1. How does taxi demand vary across space and time in Chicago, and what plausibly explains the patterns observed at different spatial units and temporal bin sizes?
2. Where are the persistent spatial hot spots of demand, and what do Gaussian mixture models and kernel density estimation reveal about their structure?
3. Can demand be predicted for a specific spatial unit and time bucket rather than simply extrapolated from the previous hour, and how does a support vector machine compare against a feedforward neural network on this task?
4. What charging policy should an electric taxi driver follow at home, given a charging rate that can be adjusted every fifteen minutes, a battery capacity that cannot be exceeded, and a next-shift energy requirement that is only known probabilistically at the moment the driver departs?
5. What do these findings imply for the client's fleet operations, and in particular, should the client invest in private home charging infrastructure or rely on public charging networks instead?

2 Data Description

2.1 Data Sources

Three raw datasets underpin the analysis: taxi trip records, hourly weather observations, points of interest, accompanied by a boundary layer of Chicago's community areas. Each is described below by origin, coverage, and native granularity; the cleaning and feature-engineering steps applied to each follow in Section 3.

The taxi trip records come from the Chicago Data Portal's Taxi Trips dataset, which publishes one row per dispatched, metered taxi ride in the city. The raw download spans January 1, 2024 through May 31, 2026 and holds 16,086,255 trips across 23 fields: trip and taxi identifiers, start and end timestamps, duration and distance, the fare broken into its components and as a total, the operating company, and pickup and dropoff locations given as census tract, community area, and centroid coordinates. Locations are masked for privacy, so a published coordinate is the centroid of the containing census tract rather than the true pickup point, and the tract itself is withheld on just over half of all trips (55.3% of pickups, 56.6% of dropoffs) against 2.8% and 8.8% for community area. Each trip is a ride requested wherever and whenever a passenger wanted one, which is the demand pattern the client's own fleet will have to anticipate.

Weather observations come from the ERA5-Land reanalysis product, pulled through the Copernicus Climate Data Store API for a single coordinate near Chicago (latitude 42, longitude -87.8) rather than a network of ground stations. The record runs from January 1, 2024 through May 12, 2026 at hourly resolution and carries six variables: 2-metre air temperature, total precipitation, snow cover, snow depth, and the two horizontal components of wind at 10 metres. A reanalysis combines numerical weather prediction models with observational data through data assimilation, resulting in a temporally continuous representation of atmospheric conditions, which no single Chicago weather station could promise.

Points of interest come from OpenStreetMap, extracted with OSMnx for the "Chicago, Illinois, USA" boundary and limited to three tag families: amenity, shop, and tourism. The raw download returns 43,237 points. Unlike the trip and weather data, this is a snapshot of the map as it stands today, not a historical record tied to the trip period. The boundary layer for Chicago's 77 official community areas is used for spatial reference and choropleth mapping throughout the report. It is a distinct unit from the census tracts already present in the trip data itself.

3 Data Preparation

This section describes the preparation pipeline that turns the raw sources described in Section 2 into the analysis-ready datasets used throughout the report. A detailed table of the engineered features can be seen in the Table 2 in the appendix.

3.1 Taxi Trip Description and Cleaning

Cleaning the raw trip table (Section 2) proceeds in three stages: trimming columns, removing unusable rows, and splitting the result into the two parallel datasets the rest of the pipeline consumes.

Seven of the 23 raw fields are dropped as redundant. The individual fare components (fare, tips, tolls, and extras) are already summed into the trip total, the payment type says nothing about where or when demand appears, and the two point-geometry columns merely repeat the latitude and longitude stored separately. Sixteen fields remain: trip and taxi identifiers, start

and end timestamps, duration and distance, pickup and dropoff census tract and community area codes, the fare total, the operating company, and pickup and dropoff coordinates. The export writes decimals with commas and prefixes currency with a dollar sign, so these columns are parsed locale-aware before any numeric comparison is made on them. From the coordinates we also derive an H3 cell index per trip at resolutions 7, 8, and 9, which gives the hexagonal units used alongside tracts and community areas later.

Rows are removed on two grounds. Records missing a taxi identifier, an end timestamp, a duration, a distance, or a fare total are dropped outright; each of these is absent on well under 0.2% of trips, a share small enough that imputation would add assumptions without adding information. A sanity filter then removes trips with a non-positive duration or distance, or a negative total, none of which describe a journey that actually happened. No statistical outlier removal is applied beyond that: extreme but plausible values, such as long airport transfers, are legitimate demand and are deliberately kept.

Because census tract is withheld far more often than community area, the cleaned data is kept as two parallel datasets rather than forcing one compromise on every downstream analysis. The larger retains every trip with a known community area; the stricter keeps only those that also carry a census tract. Both are truncated to the last hour the weather series covers, so that no model can later be trained or evaluated against forward-filled weather. That leaves **12,949,755 trips at community-area completeness** and **6,052,512 at census-tract completeness**, both spanning January 1, 2024 through May 12, 2026.

3.2 Weather Data

The final dataset comprises 17,544 hourly observations across nine columns (a timestamp, six raw variables (Table 1), and two engineered wind variables) with no missing values, consistent with the reanalysis nature of the data. A small number of precipitation and snow cover values were marginally negative due to rounding errors in the underlying computations and were clipped to zero; temperature and precipitation were additionally converted to Celsius and millimeters for interpretability. Exploratory analysis confirms clear seasonal patterns, with snow cover and snow depth peaking between November and March/April and falling to near zero in summer. Snow cover approaches 100% precisely when snow depth peaks and temperatures are lowest, confirming internal consistency, while precipitation shows no strong correlation with snow variables, as expected given their distinct physical origins. No anomalies or implausible extremes were found. The temperature ranges from below −30°C to roughly 35°C, precipitation stays below ~20 mm/hour, and snow depth does not exceed 30 cm which is consistent with Chicago's historical climate. For the client, this means the weather inputs for our models are trustworthy and reliable to use.

3.3 Points of Interest

Point-of-interest data was optional under the assignment brief, but there is precedent for treating it as a demand signal: Yan et al. (2020) predict taxi demand at city scale by combining several sources of urban geospatial data, point-of-interest records among them. We therefore include POI density to test how much it explains about where and when ride demand originates in Chicago. Coverage across the three tag families is uneven: 7,053 records lack an amenity tag, 37,552 lack a shop tag, and 41,738 lack a tourism tag, so most points carry only one of the three. We collapsed them into a single `poi_type` field by taking amenity where present and falling back to shop, then tourism, which left every point with one of 311 distinct raw types and none without a type or a geometry.

At 311 types the data is too granular to serve directly as model features, so we mapped each raw type onto one of fourteen broader categories that plausibly drive ride-hailing demand: food and drink, nightlife, entertainment, leisure and sports, grocery, shopping, health, finance, education, lodging, transport, automotive, services, and civic and community. The mapping doubles as a filter. Fifty-five raw types like infrastructure and street furniture (benches, parking entrances, post boxes, waste bins), fit no category and were dropped, leaving 20,108 categorised points of interest. Roughly half of those are building polygons rather than single points, so each was reduced to its centroid and assigned to a spatial unit, then aggregated to all three units used downstream: H3 resolution 7, census tract, and community area. The handful of points that fall just outside the official city boundary are excluded from the tract and community aggregations but retained at the H3 level, which is defined continuously over any coordinate.

3.4 Spatial and Temporal Discretisation

The grid is built from H3 resolution 7, every hexagon whose centroid falls inside the Chicago boundary. Census tract and community area join the same lookup by matching each hexagon centroid against the tract and community polygons, giving one crosswalk across all three spatial units. Every unit is then crossed with two temporal bins, hourly and daily, with pickups aggregated against all spatial and temporal levels. Spatial units that never register a single pickup over the full 2.4-year window are dropped before the grid is written; the reduction removes only all-zero rows and costs no trips.

4 Data Analytics

This section covers all analytical methods applied to the prepared data: descriptive spatial analytics, predictive modelling of demand (SVM and neural networks), and reinforcement learning for smart charging.

4.1 Descriptive (Spatial) Analytics

To better understand the characteristics of taxi demand before developing predictive models, we first perform a descriptive analysis of the dataset. The analysis investigates spatial and temporal demand patterns, examines trip characteristics, compares different spatial aggregation levels, and identifies persistent demand hot spots. ### Demand Patterns The taxi demand data exhibit strong spatial and temporal heterogeneity. Most trips are relatively short in terms of both travel distance and duration, while trip fares show a pronounced right-skewed distribution. Consequently, the majority of journeys are inexpensive, whereas a comparatively small number of long-distance trips account for substantially higher fares. Spatially, taxi activity is highly concentrated in a limited number of locations. Pickup and dropoff demand are strongly correlated, indicating that areas generating many departures also receive many arrivals. However, comparing pickup and dropoff distributions reveals localized differences that likely reflect commuting flows, airport traffic, and the separation of residential and commercial districts. Additional descriptive statistics and distribution plots are provided in **Appendix A**.

4.1.1 Spatial Patterns

Taxi demand was analysed using community areas, census tracts and H3 hexagonal grids at multiple resolutions. Although all representations reveal similar large-scale demand structures, the selected spatial resolution considerably influences the observed level of detail. Community areas provide a coarse overview of demand, while census tracts capture substantially finer local

variation. H3 resolution 7 offers a compromise between spatial detail and robustness, whereas H3 resolution 8 largely reproduces the census tract representation. This behaviour results from the privacy-preserving coordinate masking applied to the original dataset, where published coordinates correspond to census tract centroids whenever available. [**Figure X: Spatial demand distribution**] Across all spatial resolutions, demand remains highly concentrated in only a small number of regions. Increasing spatial resolution improves the visibility of localized demand patterns but simultaneously introduces considerable sparsity, with many spatial units containing few or no observations during large parts of the observation period. Further spatial comparisons, concentration measures, Lorenz curves and pickup-dropoff difference maps are shown in **Appendix A**.

4.1.2 Temporal Patterns

Taxi demand follows clear and recurring temporal patterns. Demand is lowest during the early morning hours, increases throughout the day and reaches pronounced peaks during commuting and evening leisure periods. Distinct differences are also observed between weekdays and weekends, reflecting changes in travel behaviour. [**Figure X: Average hourly taxi demand**] Besides the intraday cycle, demand also varies systematically across weekdays and seasons, indicating that temporal information is highly informative for subsequent forecasting models. Additional temporal analyses, including weekday-weekend comparisons and seasonal demand patterns, are provided in **Appendix A**.

4.1.3 Effect of Spatio-Temporal Resolution

Abstimmen mit predicive

4.1.4 Demand Hot Spots

Persistent demand hot spots were identified using Gaussian Mixture Models (GMM) and Kernel Density Estimation (KDE). Both approaches consistently identify the central business district and major transportation hubs as the dominant centres of taxi activity. [**Figure X: Gaussian Mixture Model hot spots**] The probabilistic clustering produced by the GMM groups regions with similar spatial demand characteristics, while KDE provides a continuous representation of demand intensity. Both methods highlight areas associated with dense commercial activity, transportation infrastructure and entertainment facilities, demonstrating the strong influence of land use on taxi demand. The complementary KDE visualisation is presented in **Appendix A**.

4.2 Predictive Analytics (SVM & Neural Networks)

Fleet planning needs to know where and when demand will appear before a shift starts. We predict trip starts per spatial unit and time bucket from the cell's identity and location, calendar, weather, and POI features alone, a scenario question such as expected demand on a rainy Sunday morning on the outskirts, not an extrapolation from the previous hour. Two model families are compared: a Support Vector Machine, fitted directly on demand and on the residual over a historical-mean baseline, and a feedforward neural network. Both train across six grids crossing three spatial units (H3 resolution 7, census tract, community area) with hourly and daily bins, and both are scored against the same historical-mean baseline, letting us ask whether a learned model, and a deep one especially, earns its added complexity.

The target is `Total_Trip_Start`, the trip count per spatial unit and time bucket. Train, validation, and test are cut along the calendar rather than drawn at random: the earliest 50% of unique dates train the models (2024-01-01 to 2025-03-06), the next 20% validate (2025-03-07 to 2025-08-26), and the final 30% are held out for testing (2025-08-27 to 2026-05-12). A random row-level split would leak autocorrelated demand into the test set and contaminate `base_demand`. Model selection runs on validation; test is scored once, for the selected models. The reference point throughout is a historical mean per spatial unit, keyed by month and day of week, plus hour on the hourly grids. Computed on the training split alone and merged into validation and test: as the `base_demand` feature available to both models, and as the skill-score benchmark, `1 - MAE_model / MAE_baseline`, making raw errors incomparable between them.

4.2.1 Support Vector Machine

Two Support Vector Machines are trained, differing only in what they aim at. The direct model predicts demand outright. The residual model predicts something narrower: how far a given place and time departs from its own historical average, which is then added back to that average. The reason to bother is that the historical average already carries most of the predictable signal, so a model built to correct it can spend its effort where the average falls short rather than relearning what is already known.

Both share the same training setup. Because most place-and-time cells are empty, each model trains on a balanced mix of busy and empty cells so it does not simply learn to predict "nothing", while validation and testing keep the true, mostly-empty proportions the models will face in practice. For each model we compare four feature sets, from a calendar-and-location baseline up to one that adds weather and points of interest, and search for the best model settings. The exact search grid and feature blocks are listed in Table 3 and Table 4.

4.2.1.1 Validation Results

Predicting demand outright, the direct model rarely earns its keep. Across nearly every grid its tuned winner trails the historical-mean baseline on validation, and the one grid where it comes out ahead does so by a margin too thin to call a win. The pattern follows from what the model is asked to do: reconstructing absolute demand from scratch means relearning the historical pattern before it can improve on it, and most of the six grids do not hand it enough signal to do both (Table 5, Figure 6).

Reframing the target as a correction to that same historical mean, rather than demand itself, changes the picture everywhere, but not evenly. One grid delivers a clear win (Community/Daily), a second, also on daily data, adds a smaller but genuine edge (H3_7/Daily). Three more land close enough to the baseline, in either direction, to call them ties; and the most zero-inflated grid (Census/Hourly) is the one place the correction does more harm than good. Daily grids beat their hourly counterparts throughout, under both framings, coarser data giving each more to hold on to (Table 6, Figure 8, Table 7).

The two strongest configurations overall are both residual models fitted on daily data, and both do well with a simple linear fit rather than a more complex one. The margin they add over the historical average is real but modest, with the exact figures in Table 6.

4.2.2 Deep Learning Model

The neural network differs from the Support Vector Machine in two ways that matter. It trains on the full data at its natural, mostly-empty proportions, because a balanced sample

actively hurt its accuracy in early tests. And it uses a single feature set at every level, the fullest one, rather than comparing four, a compute-budget choice rather than a finding that the others do not matter. As with the Support Vector Machine, the network's depth and training settings are chosen by a search per level and selected on the validation data; the winning configurations are listed in Table 8.

4.2.2.1 Validation Results

The network is best read as an hourly specialist rather than an all-purpose forecaster. It clears the historical-mean baseline at only two of the six levels, both hourly, and its one clear win, a 9.7% gain over the baseline at community/hourly, sits at the same level and grain where the Support Vector Machine is weakest. Every daily-resolution grid falls short of that same naive baseline regardless of spatial detail, community included and h3_7/daily worst of all at nearly 32% below it, so the temporal grain the model predicts at matters far more than which spatial unit it covers (Table 9, Figure 10).

The winning architecture tracks that same pattern rather than following spatial density in any simple way. The strongest level, community/hourly, uses the largest and deepest network tested; the two census levels, the sparsest and most zero-inflated in the dataset, both settle on the smallest. Dropout is selected only at the two levels most prone to overfitting, h3_7/daily and census/daily, consistent with those being the hardest grids to fit without memorizing noise (Table 8). Taken together with the Support Vector Machine's own results, the pattern points at the data rather than either model: a historical average already captures most of what a day or an hour's demand will look like, and the daily grids in particular leave the network little room to add anything beyond replaying that average back.

4.2.3 Model Comparison

The two model families are compared directly on the same held-out test data, at the level where each independently performs best: Chicago's 77 community areas, assessed separately at daily and hourly resolution. That both the Support Vector Machine and the neural network converge on this same, comparatively coarse geography is worth stating plainly on its own: neither the finer hexagon grid nor individual census tracts produced a more accurate forecast for either model. A citywide rollout does not need finer-grained location data to plan around.

No single model wins outright. Instead, the two split the job by time horizon. Forecasting a full day ahead, the Support Vector Machine cuts the average error from 49.58 to 46.34 trips per area per day, a 6.5% improvement over simply assuming tomorrow looks like the recent average; the neural network, over the same day-ahead horizon, does 10.0% worse than that same naive assumption. Forecasting within a single hour, the ranking reverses: the neural network cuts the average error from 2.90 to 2.62 trips per area per hour, a 9.7% gain, while the Support Vector Machine is essentially indistinguishable from the naive estimate (Table 11). For the client, the practical reading is direct: trust the Support Vector Machine for next-day capacity planning, and trust the neural network for forecasting how demand moves within a shift.

That division sharpens once demand is broken down by how busy an area actually is (Table 13). At the daily level, the Support Vector Machine beats the naive average across every demand tier tested, from pickup-free days to the busiest area-days on record, and its edge is largest, 7.0%, on exactly the days that matter most financially: the high-demand days where too few vehicles means turning riders away and too many means idle, charging capital sitting unused. The neural network trails the baseline across every one of the same daily tiers, worst of all on medium-demand days, where it gives up more than 30% of the baseline's accuracy. Within

the hour, the Support Vector Machine's record is more mixed: a 19.4% gain on hours with no demand at all, roughly break-even on moderately busy hours, and a loss of ground on both quiet, low-demand hours and, more consequentially, the busiest ones. The neural network beats the baseline on every hour with any demand at all, gaining 11.3% on medium-demand hours and 11.1% on high-demand hours.

A forecast that only holds up in good weather would be of limited use to an operator whose founding question is what to expect on a rainy Sunday. Neither recommended model shows that weakness. The Support Vector Machine's day-ahead accuracy is, if anything, slightly better on rainy days than dry ones, and the neural network's hourly accuracy is marginally stronger in the rain too. Weather is not a blind spot for either tool at this level of geography.

Mapped across the city, each model's advantage falls exactly where the demand-tier results predict (Figure 15). At daily resolution, the Support Vector Machine's lower error holds in almost every district. At hourly resolution, the neural network's advantage concentrates in the busiest central districts, downtown and the Near North Side, where hour-to-hour demand swings are largest and a positioning mistake is most expensive. The underlying maps, and a further breakdown by day of week, hour of day, and calendar period, are collected in Section A.

4.2.3.1 Recommendation

The two models are best deployed together rather than chosen between: the Support Vector Machine for day-ahead capacity planning across service areas, and the neural network for repositioning vehicles within a shift once the operation is ready for it. Because a finer geographic grid bought neither model any extra accuracy, near-term effort is better spent getting day-ahead district planning right than on acquiring more granular location data.

4.2.4 Shortfalls and Improvement Levers

Both models share one ceiling. On the sparsest, most fragmented grids, too few trips occur for either to add much beyond the historical average, and more broadly that average already captures most of what is predictable about a given place and time. Neither model replaces it so much as improves on it at the margin. The clearest lever for a follow-up is to train future versions on demand as it actually occurs, rather than on the balanced mix used here to cope with the many empty cells; that is the likeliest route to closing the Support Vector Machine's remaining weakness on the busiest hours.

4.3 Smart Charging (Reinforcement Learning)

The final task moves from forecasting demand to acting on it: an automated controller that decides how fast an electric taxi should charge at home between shifts, balancing the electricity bill against the risk of leaving too little charge for the next day's work. We compared two reinforcement-learning methods, Tabular Q-Learning and a Deep Q-Network (DQN). The simpler Tabular approach is the one we recommend deploying now; DQN is held back for later, more complex versions of the problem, for the reasons that follow.

4.3.1 Problem Framing & Methodology

Each vehicle returns home at 2:00 PM and must depart at 4:00 PM. During this window, an automated controller decides every 15 minutes how fast to charge the battery, choosing from four charging speeds. Electricity prices rise later in the afternoon, while the energy required

for the day's shift is unknown until departure and varies around 30 kWh. Running out of energy mid-shift causes a missed fare and a stranded vehicle is substantially more costly than a marginally higher electricity bill. Thus, the controller must balance cost against this risk. The problem was formalized as a Markov Decision Process (MDP). At each step, the model observes battery level and remaining time, and selects one of four charging speeds (0, 7.33, 14.67, or 22 kW). Each decision incurs a small electricity cost, and a $500 penalty applies if the battery is insufficient at departure. This penalty was set far above the maximum achievable savings from smart charging (at most a few cents per day) so that the model always prioritizes reliability. The arrival battery level is drawn from a uniform distribution (0–50% of capacity) rather than assumed as always empty, reflecting realistic driver behavior and creating a genuine cost-risk trade-off. The per-kWh price coefficient rises from ~7 to ~11 cents across the eight time steps, incentivizing earlier charging where safe. Two methods were compared: Tabular Q-Learning, a lookup table well suited to this problem's small, enumerable state space, and DQN, which replaces the table with a neural network capable of generalizing across states. Each method was trained independently five times with different random seeds and evaluated on several thousand simulated days per run, to avoid drawing conclusions from a single, potentially lucky training outcome.

4.3.2 Results

4.3.2.1 Tabular Q-Learning

Training over 150,000 simulated days shows the training score improving and then stabilizing, with the failure rate dropping sharply to a low, stable level (Figure 22, Figure 23), indicating convergence to a consistent policy. The learned policy (Figure 24) uses both intermediate charging speeds across a substantial share of states rather than only the extremes, indicating a genuinely differentiated strategy. Tabular Q-Learning reduced the average daily cost from $1.92 to $1.50 ± $0.02 while keeping the failure probability (0.11% ± 0.09%) close to the naive baseline (Table 14). Overall, a roughly 22% cost reduction without a measurable increase in stranded-vehicle risk.

4.3.2.2 Deep Q-Network

DQN's training curves (Figure 25, Figure 26) oscillate throughout the full 20,000-episode period rather than stabilizing, so the best-performing weights observed at any point were retained rather than the final ones. Increasing episodes further did not help, which is also why DQN required far fewer episodes than the tabular approach (20,000 vs. 150,000), reducing the compute time needed drastically. The retained policy (Figure 27) charges at full power in almost all states, differing from the naive baseline in only two narrow state regions. Largely reproducing the naive strategy rather than a materially different one. The DQN heatmap appears fully populated while the Q-table heatmap only covers a diagonal band, because the Q-table stores entries only for visited states, whereas the neural network returns an output for any input, including states never encountered during training. DQN reduced the average daily cost to $1.46 ± $0.24 with a failure probability of 1.16% ± 0.96% which is comparable on average to Tabular Q-Learning, but with much greater variability across runs (Table 15). The per-seed variation is one of the key findings: one run reproduced the naive strategy almost exactly, while the other four achieved lower costs at markedly higher failure rates (Table 16). This inconsistency, more than the average result, is why DQN is not recommended for production use at this stage, as we are able to achieve comparable or even better results with noticeably less effort.

5 Discussion and Outlook

Taxi demand across Chicago is concentrated, not spread. At every spatial resolution tested, roughly 99% of pickups occur in the busiest tenth of locations, and hot-spot analysis places that concentration where the densest mix of commerce, transit, and evening entertainment sits: the central business district and the major transport hubs. Demand also follows a dependable daily rhythm, quiet overnight and peaking around the commute and again in the evening, with weekends distinct from weekdays. This is structure a fleet operator can plan against. Moving to a finer geographic grid added little beyond what census tracts and community areas already showed, because the data's privacy masking snaps most coordinates back to their census tract.

For the client, that points to concentrating the initial fleet in the few districts that generate almost all demand rather than spreading vehicles evenly, and to planning around the administrative geography Chicago already publishes rather than a bespoke fine-grained grid.

Demand can be predicted for a specific place and time, not just extrapolated from the previous hour, though within limits. Both models are most accurate at the same geography, Chicago's 77 community areas, and neither wins outright; they divide the work by time horizon. The support vector machine is the more reliable day-ahead forecaster, beating a historical-average benchmark by about 6.5% and holding that edge on quiet and busy days alike. The neural network is stronger within the hour, beating the same benchmark by about 9.7%, with its advantage concentrated on the busy hours where the support vector machine slips. Neither falters on rainy days, the rainy-Sunday scenario the brief singled out. The deep model does not earn its added complexity everywhere: at day-ahead resolution it trails even the naive benchmark. The two are best run together, the support vector machine sizing the fleet a day out and the neural network repositioning it within a shift once the operation is ready.

For smart charging, Tabular Q-Learning is the method to deploy first. It converged reliably across every training run and cut the average daily charging cost by roughly 22% against the naive baseline, while holding the risk of running out of charge near that baseline and so keeping vehicle availability intact. The Deep Q-Network matched it on average cost and failure rate but varied far more from run to run, a poor trade for a service where one stranded vehicle means a missed fare. It becomes worth revisiting only as the fleet and its charging decisions grow more complex.

That same controllability settles the larger infrastructure question. Both the cost saving and the reliability depend on dictating when and how fast each vehicle charges, which is only possible on chargers the operator controls. Private charging at the driver's home, over the idle window between shifts, is therefore the better default than leaning on public stations, where the driver pays the posted rate and competes for an open plug. Public charging is best reserved as a fallback, for unusually long shifts or drivers without home access. This rests on the operational case rather than a full comparison of installation cost against public-charging fees, which a follow-up should price out.

These findings are directional, not final. They come from Chicago's existing taxi trade rather than the client's own electric fleet, and the demand models beat a simple historical average only modestly, so location and calendar already explain most of what the data can. The charging result likewise rests on a fixed daily price curve and a single vehicle, not a full fleet in a live market. Validating both against the client's own operating data, once it exists, would do more to confirm these conclusions than any further modelling.

A Additional Figures and Tables

This appendix collects supporting figures and tables that are not reproduced in the main body of the report, organized by the notebook that produces them (see the pipeline overview in the project `CLAUDE.md`).

A.1 Taxi Data Preparation (1.1)

No additional labeled figures or tables from this notebook.

A.2 Weather Data Preparation (1.2)

Table 1: Selected Meteorological Variables

Variable	Description	Datatype
2m Temperature (K and °C)	Air temperature at 2 meters above the surface. Expected to drive customer demand (e.g., fewer walk/bike substitutes in extreme cold) and EV battery performance, since range may drop at low temperatures.	float64
Total Precipitation (m and mm)	Accumulated liquid and solid precipitation reaching the surface. Expected to increase short-distance ride demand and reduce walking/cycling alternatives.	float64
10m Wind Components u10 and v10	Eastward and northward horizontal wind components at 10 meters above ground level. Combined into wind speed and direction (see Feature Engineering); relevant for extreme-weather demand spikes and outdoor-wait discomfort.	float64
Snow Cover / Snow Depth	Fraction of the grid cell covered by snow, and mean snow thickness on the ground (excluding canopy snow). Expected to directly affect road conditions, trip duration, and safety-related demand drops, and are key drivers of charging infrastructure planning in winter months.	float64

No additional labeled figures or tables from this notebook.

A.3 Points of Interest Data Preparation (1.3)

A.4 Temporal-Spatial Discretization (1.4)

No additional labeled figures or tables from this notebook.

A.5 Descriptive Analysis: Spatial (2.1)

No additional labeled figures or tables from this notebook.

A.6 Descriptive Analysis: Temporal (2.2)

No additional labeled figures or tables from this notebook.

A.7 Descriptive Analytics: Weather (2.4)

No additional labeled figures or tables from this notebook.

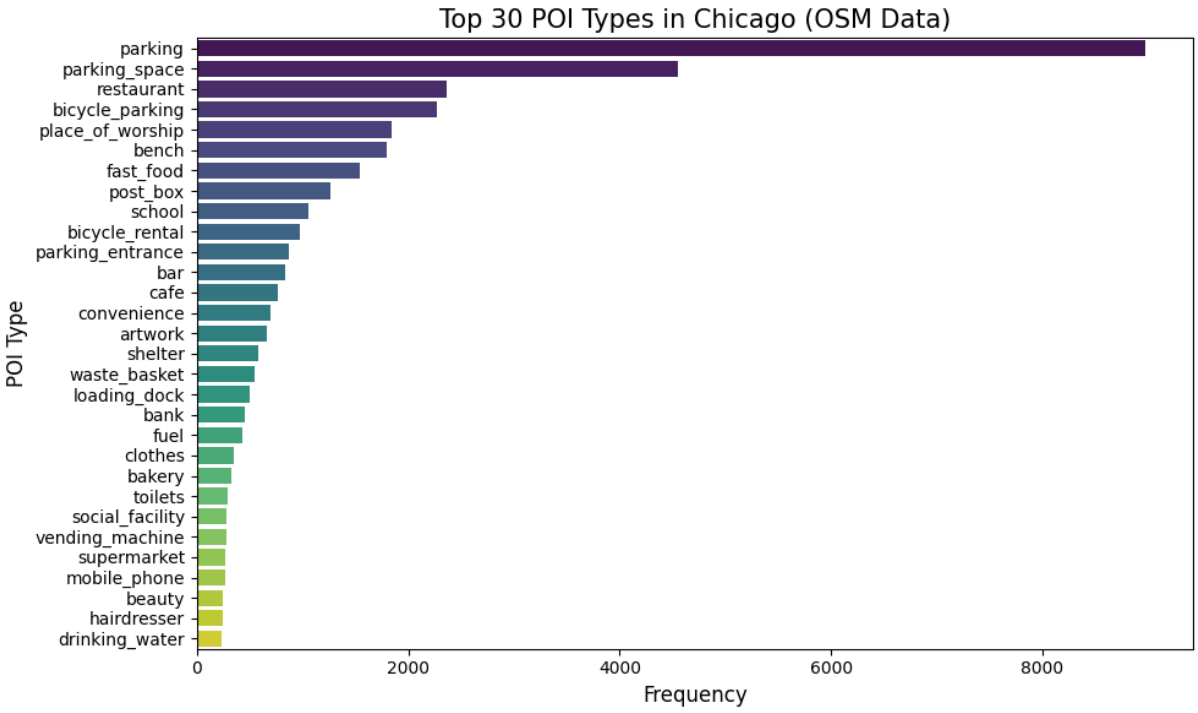


Figure 1: Top 30 most frequent raw OSM POI types in Chicago, before category mapping.

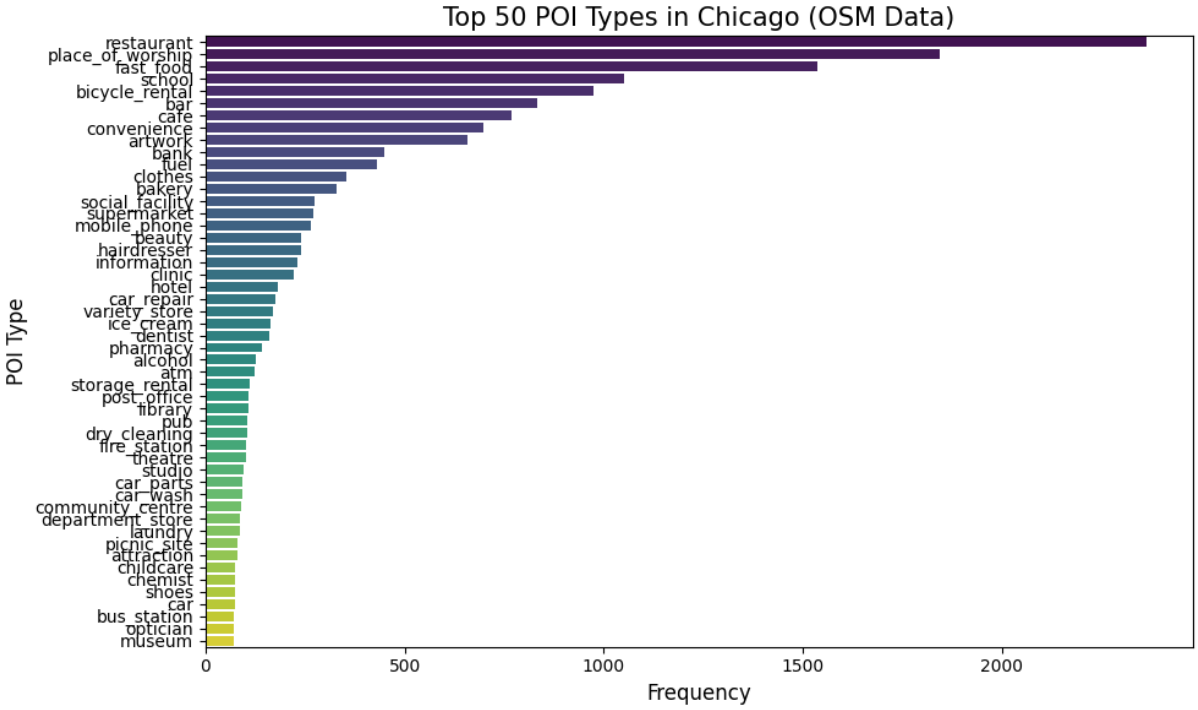


Figure 2: Top 50 most frequent POI types remaining after category filtering (unmapped 'other' types dropped).

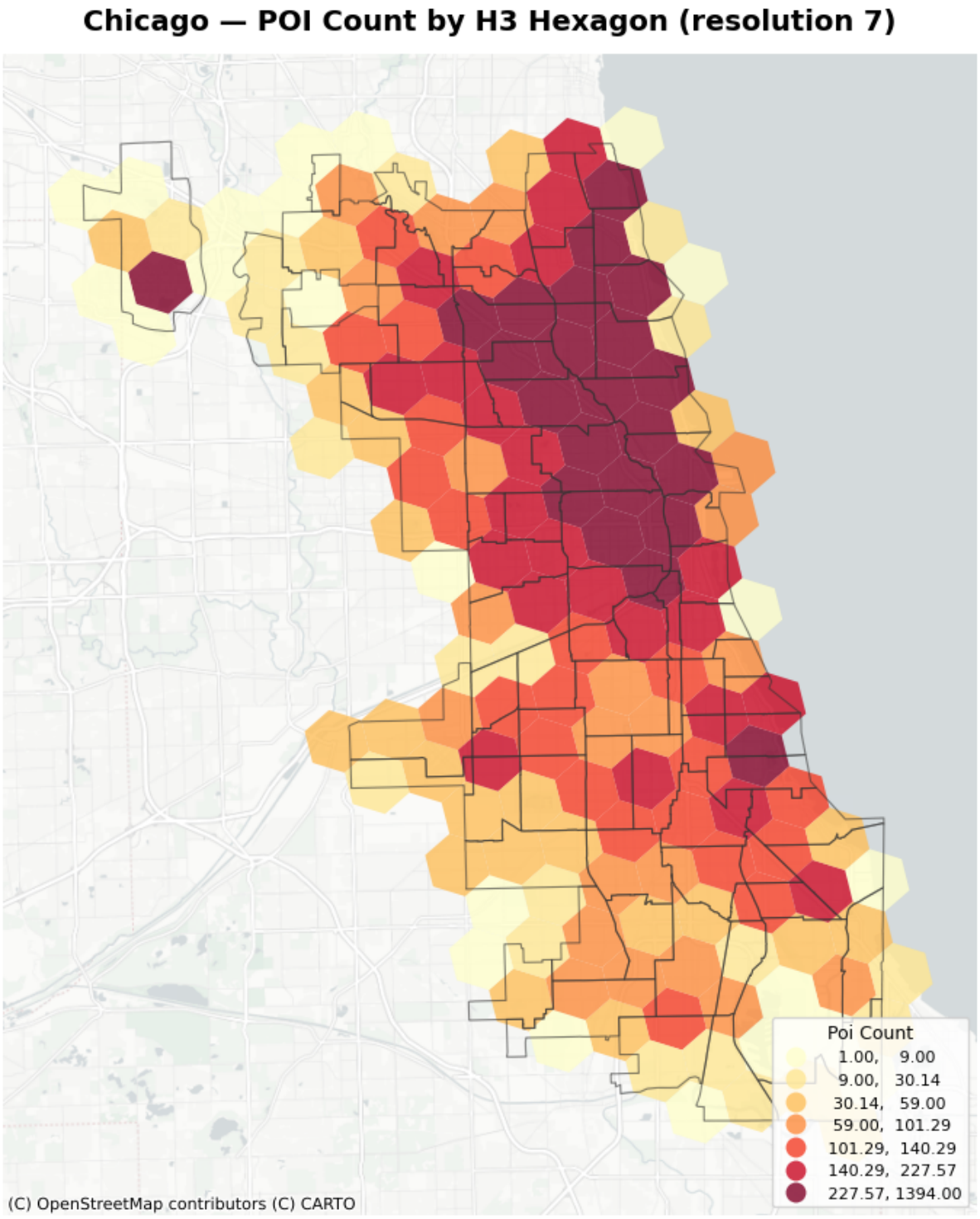


Figure 3: Total POI count per H3 hexagon (resolution 7).

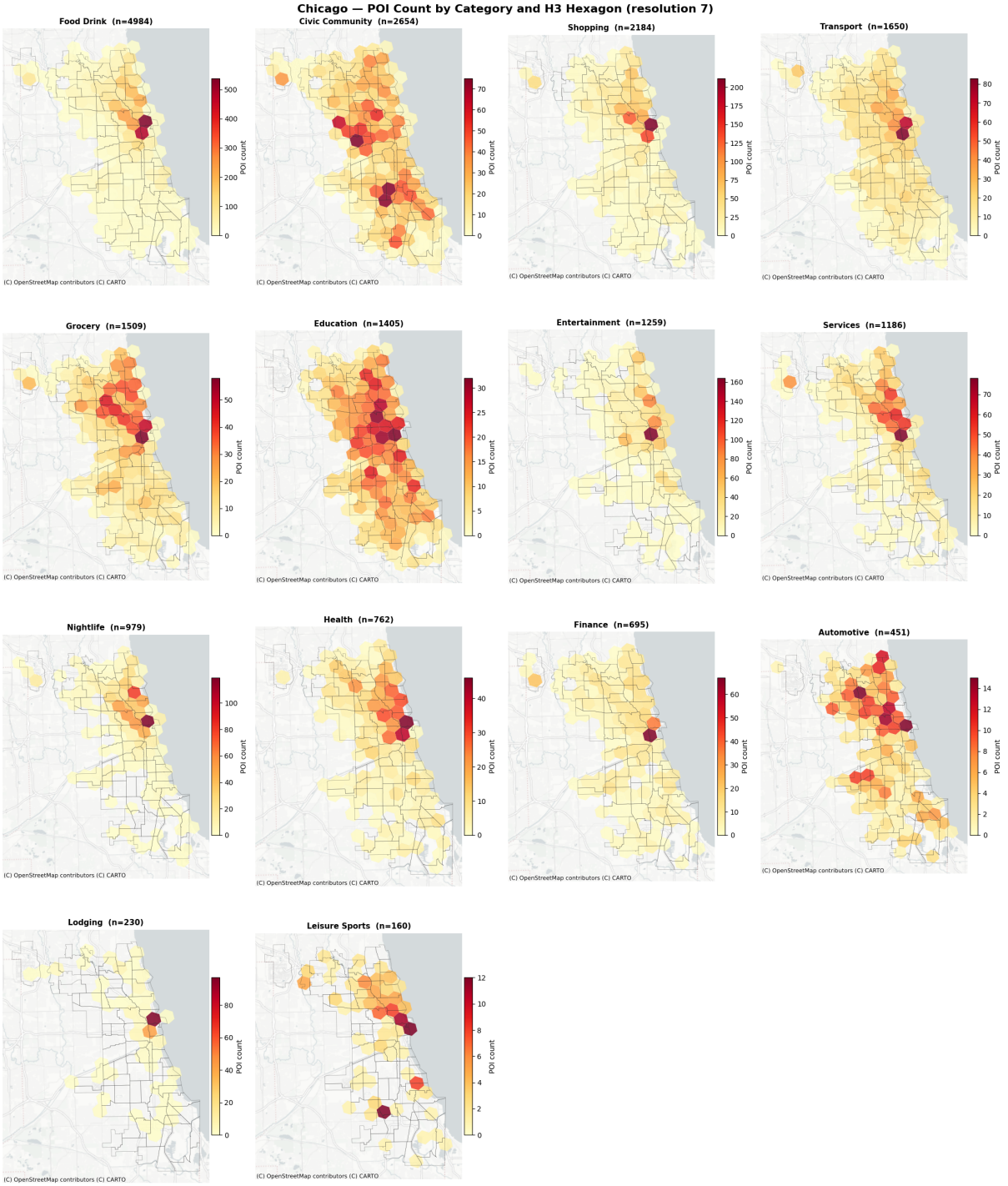


Figure 4: POI count per H3 hexagon (resolution 7), faceted by category.

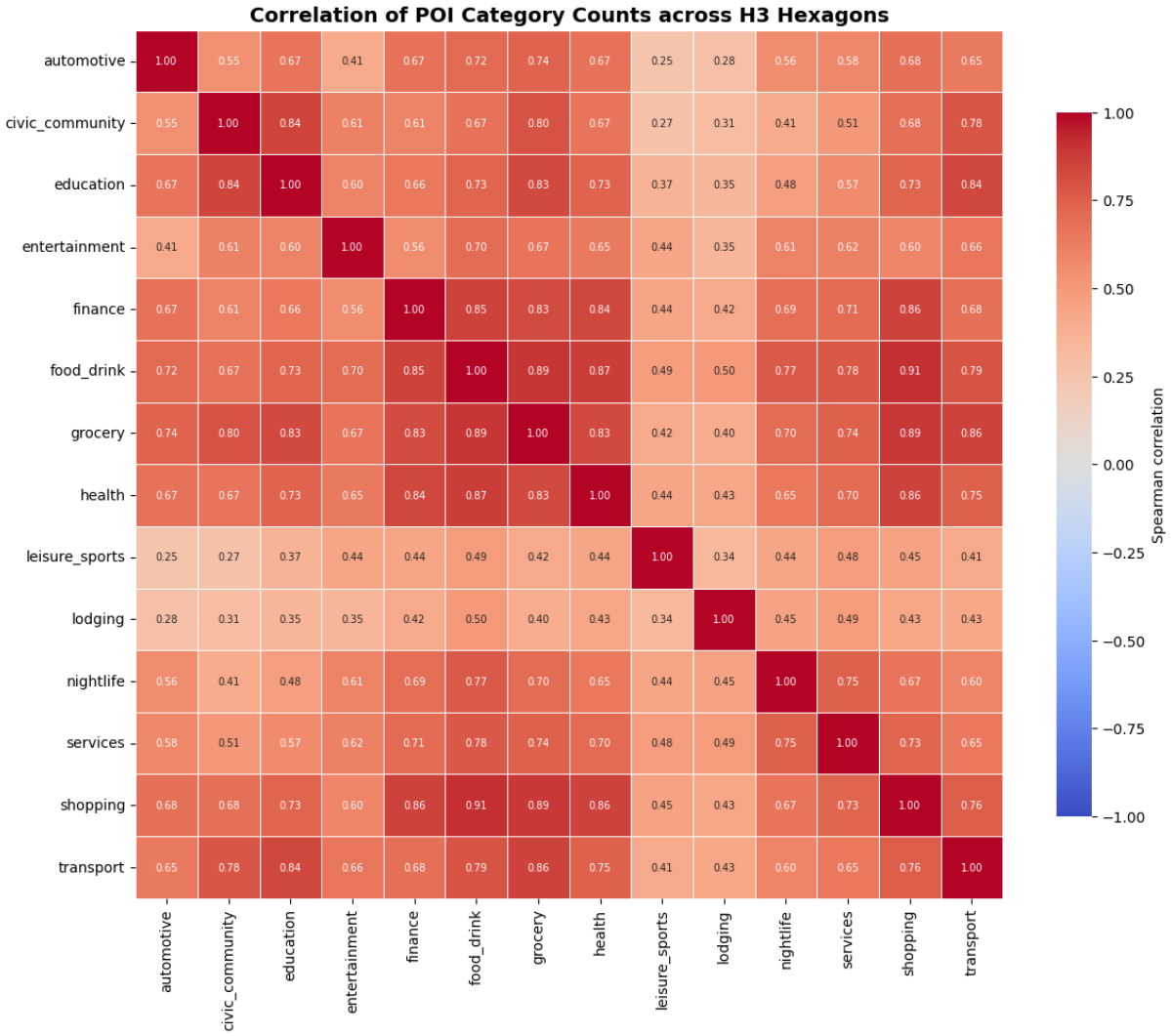


Figure 5: Spearman correlation of POI category counts across H3 hexagons (resolution 7).

A.8 Feature Engineering (3.1)

Table 2: List of all Engineered Features.

Engineered Feature	Description	DataType
distance_to_loop	Great-circle (haversine) distance from each grid cell's coordinates to the Chicago Loop (downtown reference point), used as a proxy for centrality/urban density.	float64
wind_speed	Magnitude of the horizontal wind vector, computed as sqrt($u10^2$ + $v10^2$) from the 10m u/v wind components.	float64
wind_dir	Meteorological wind direction (direction the wind blows from), computed via atan2(u10, v10) and converted to compass degrees (0-360°).	float64
is_holiday	Binary indicator flagging official U.S. federal holidays observed in Illinois, expected to shift ride-hailing demand relative to regular weekdays.	int64
is_near_holiday	Binary indicator flagging the calendar day immediately before or after a public holiday, capturing anticipatory or spillover demand effects.	int64
hour_of_day	Hour extracted from the timestamp, capturing intraday demand cycles. Only present in hourly-resolution grids.	int64
hour_sin	Sine transformation of hour_of_day, encoding time of day on a continuous circular scale. Only present in hourly-resolution grids.	float64
hour_cos	Cosine transformation of hour_of_day, paired with hour_sin to avoid the discontinuity between hour 23 and hour 0. Only present in hourly-resolution grids.	float64
month	Calendar month extracted from the timestamp, capturing seasonal variation.	int64
month_sin	Sine transformation of month, encoding the annual cycle continuously.	float64
month_cos	Cosine transformation of month, paired with month_sin to avoid the discontinuity between December and January.	float64
day_of_week	Numeric day of week extracted from the timestamp (0 = Monday, ..., 6 = Sunday), capturing weekly demand patterns.	int64
is_weekend	Binary indicator flagging Saturday and Sunday (day_of_week >= 5), expected to correlate with different ride purposes (leisure vs. commute).	int64
weekday	Numeric day of week extracted from the timestamp; identical in definition to day_of_week (0 = Monday, ..., 6 = Sunday).	int64
season	Meteorological season derived from month, encoded as an integer (1 = Winter, 2 = Spring, 3 = Summer, 4 = Autumn), capturing broader seasonal demand and weather regimes.	int64
season_sin	Sine transformation of season, representing the seasonal cycle continuously.	float64
season_cos	Cosine transformation of season, paired with season_sin to avoid a discontinuity between the last and first season of the year.	float64
base_demand	Historical average of Total_Trip_Start, computed on the training split only and grouped by spatial cell, month, day_of_week, and (for hourly grids) hour_of_day; unseen key combinations are filled with the global training mean.	float64

A.9 SVR Hyperparameter Search Grid (3.2)

Both SVR notebooks share the identical two-stage search defined in `3.2 PredictionCommonGround.ipynb`, so the direct and residual results stay directly comparable. The grid search runs only for the kernel that wins the scan, not for all three.

Table 3: Two-stage SVR hyperparameter search grid, shared by the direct (`3.2a`) and residual (`3.2b`) models.

Stage	Setting	Values searched
Kernel scan	Kernels	`linear`, `poly`, `rbf`
	Held fixed during the scan	`C = 1`, `gamma = "scale"`
	Selection criterion	Validation R^2 in count space
	Training sample	4,000 rows per class (8,000 total)
Grid search	`C`	1, 10, 50
	`gamma`	`"scale"`, 0.05, 0.2
	`degree` (polynomial kernel only)	2, 3
	Cross-validation	3-fold
	Selection criterion	Count-space MAE
	Training sample	8,000 rows per class (16,000 total)
Feature sets	Candidates	`basic`, `basic+poi`, `basic+weather`, `basic+poi+weather`
	Selection criterion	Validation R^2 in count space
Scoring	Validation subsample	50,000 rows; the level's winner is re-scored on the full validation split

The four candidate feature sets are built from the same blocks at every level. `basic` is hour (hourly grids only) plus calendar, location and history; the other three add the POI and weather blocks to it.

Table 4: Feature blocks composing the four candidate sets. Resulting widths are 10 / 24 / 15 / 29 features on daily grids and 12 / 26 / 17 / 31 on hourly grids, for `basic` / `basic+poi` / `basic+weather` / `basic+poi+weather` respectively.

Block	Features	Count
Hour (hourly grids only)	`hour_sin`, `hour_cos`	2
Calendar	`month_sin`, `month_cos`, `is_weekend`, `is_holiday`, `is_near_holiday`, `day_of_week`	6
Location	`lat`, `lon`, `distance_to_loop`	3
History	`base_demand`	1
POI	14 `poi_cat_*` category counts	14
Weather	`2m_temp_c`, `total_precip_mm`, `snow_cov`, `snow_depth`, `wind_speed`	5

A.10 SVR: Direct Model (3.2a)

Table 5: Winning SVR per level, with skill against the historical-mean baseline.

	spatial	temporal	feature_set	kernel	base_MAE	val_MAE	val_RMSE	val_R2	skill_MAE	skill_RMSE
0	h3_7	daily	basic+poi	rbf	15.8295	15.7997	88.8310	0.9494	0.0019	0.0069
1	community	daily	basic+poi	rbf	36.1879	37.4429	139.8964	0.9578	-0.0347	-0.0769
2	census	daily	basic+poi	rbf	3.8284	4.0758	27.3988	0.9341	-0.0646	-0.0354
3	community	hourly	basic+poi	rbf	2.5250	2.7382	10.3958	0.9133	-0.0844	-0.1556
4	h3_7	hourly	basic	rbf	0.8861	0.9858	6.2626	0.9148	-0.1125	-0.0348
5	census	hourly	basic+poi+weather	rbf	0.2438	0.3302	2.0741	0.8795	-0.3547	-0.0468

A.11 SVR: Residual-on-Baseline Model (3.2b)

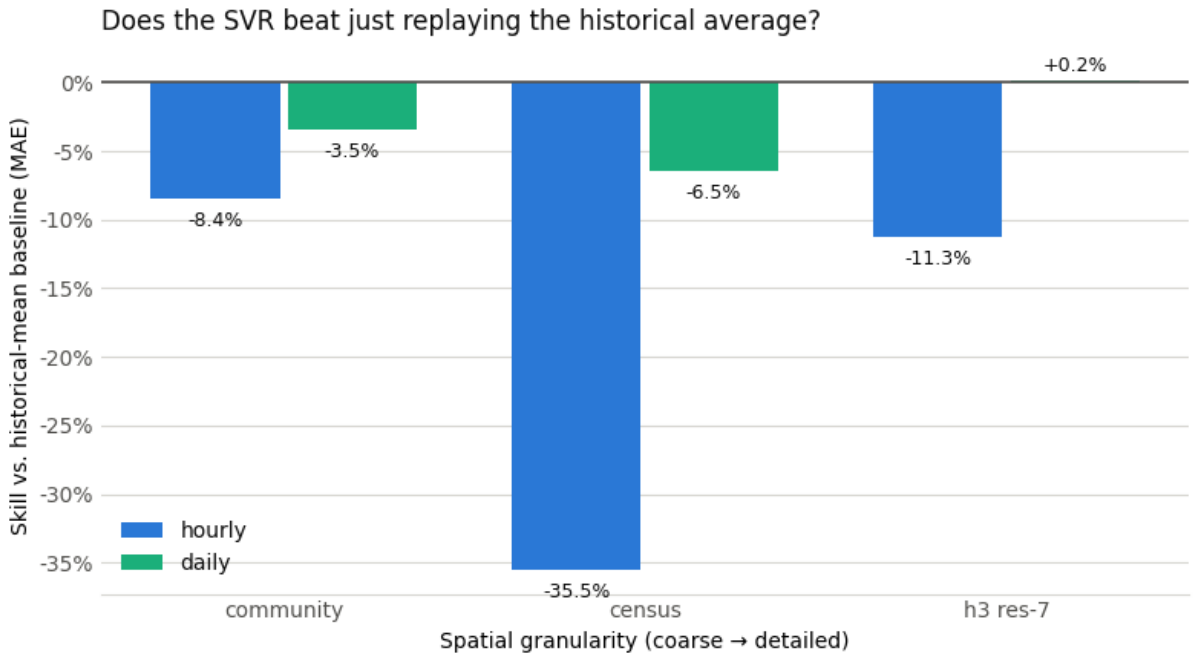


Figure 6: SVR skill against the historical-mean baseline, by spatial and temporal granularity.

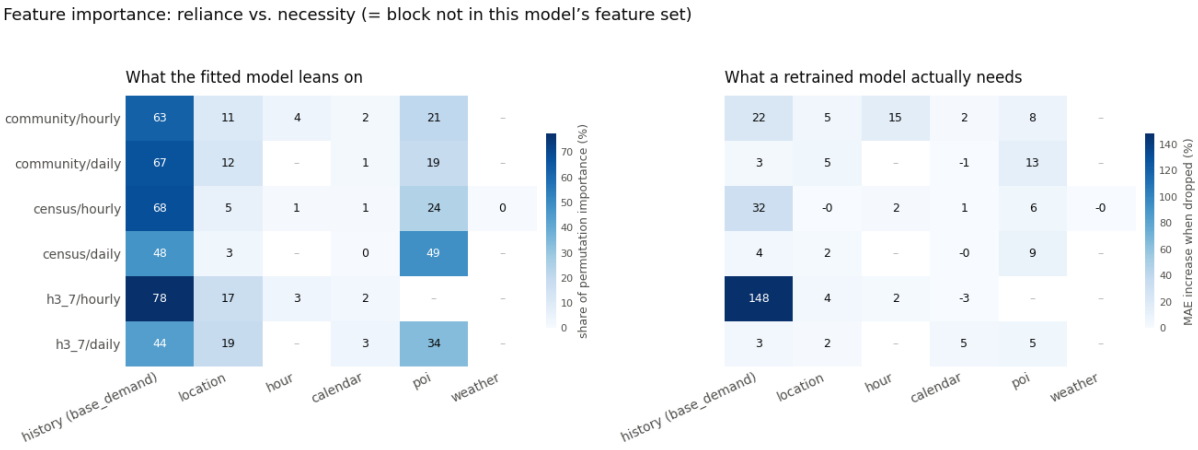


Figure 7: Left: what the fitted model leans on (grouped permutation importance, as each block's share of the level's total). Right: what a retrained model actually needs (drop-column ablation, % increase in MAE when the block is removed). The two disagree sharply on base_demand — the fitted model depends on it almost entirely, yet a model refitted without it loses very little, because lat/lon/hour/day-of-week can reconstruct it.

Table 6: Winning residual-on-baseline SVR per level, with skill against the historical-mean baseline.

	spatial	temporal	feature_set	kernel	base_MAE	val_MAE	val_RMSE	val_R2	skill_MAE	skill_RMSE
0	community	daily	basic+weather	linear	36.1879	34.2541	118.3257	0.9698	0.0534	0.0891
1	h3_7	daily	basic+poi	linear	15.8295	15.6367	84.9372	0.9538	0.0122	0.0504
2	census	daily	basic+poi+weather	rbf	3.8284	3.8174	26.1691	0.9399	0.0029	0.0111
3	h3_7	hourly	basic+weather	linear	0.8861	0.8838	5.5285	0.9336	0.0026	0.0865
4	community	hourly	basic+poi+weather	rbf	2.5250	2.5498	9.2293	0.9316	-0.0098	-0.0259
5	census	hourly	basic	linear	0.2438	0.3221	1.9674	0.8915	-0.3214	0.0071

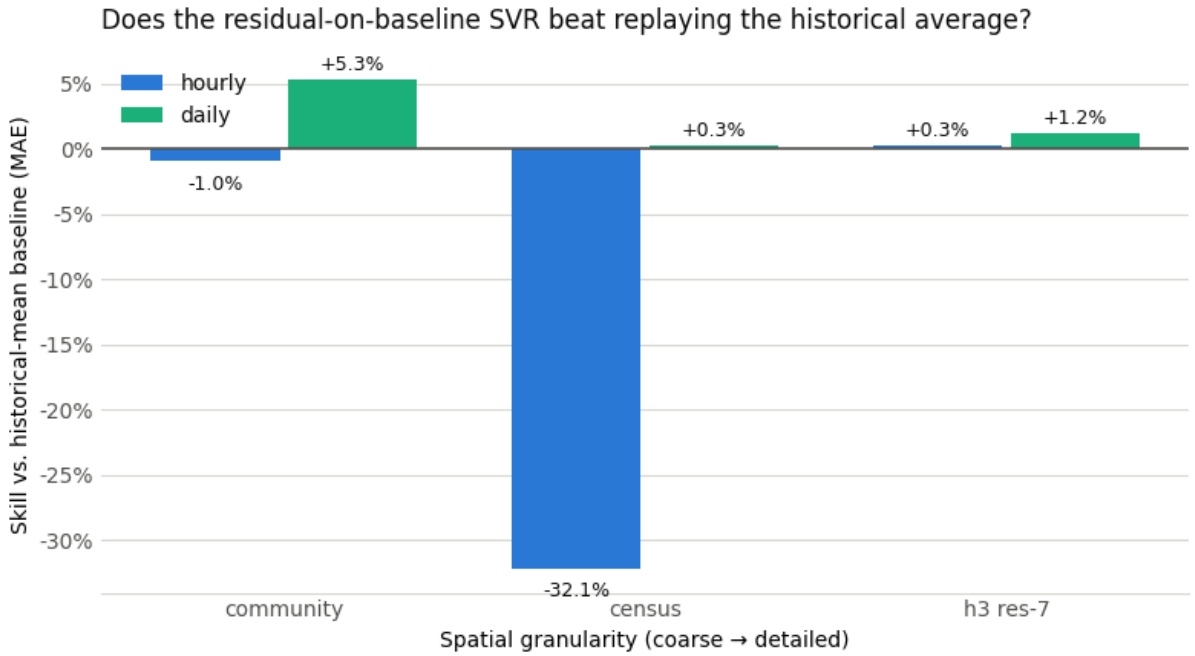


Figure 8: Residual-on-baseline SVR skill against the historical-mean baseline, by spatial and temporal granularity.

Table 7: Direct SVR (3.2a) vs. residual-on-baseline SVR trained from scratch (this notebook), per level, on the validation split.

	spatial	temporal	feature_set	kernel	skill_MAE_direct	skill_MAE_residual	delta_skill_MAE
0	h3_7	hourly	basic+weather	linear	-0.1125	0.0026	0.1151
1	community	daily	basic+weather	linear	-0.0347	0.0534	0.0881
2	community	hourly	basic+poi+weather	rbf	-0.0844	-0.0098	0.0746
3	census	daily	basic+poi+weather	rbf	-0.0646	0.0029	0.0675
4	census	hourly	basic	linear	-0.3547	-0.3214	0.0333
5	h3_7	daily	basic+poi	linear	0.0019	0.0122	0.0103

A.12 Deep Learning Prediction (3.3)

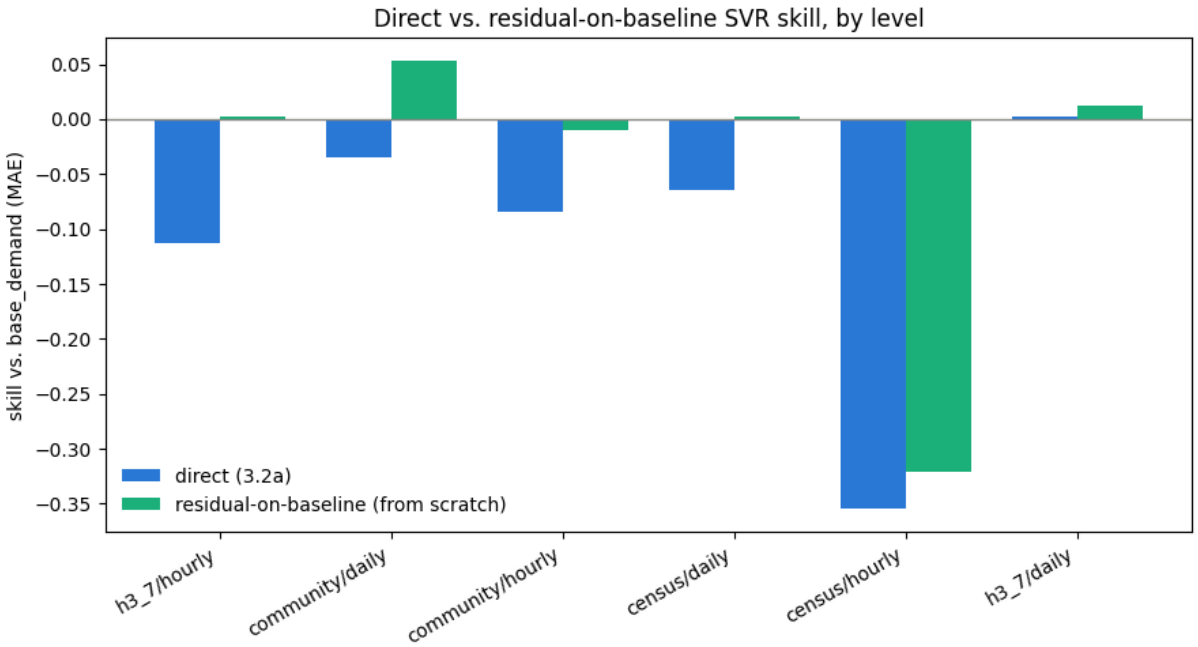


Figure 9: Direct (3.2a) vs. residual-on-baseline (this notebook) SVR skill, both independently tuned, per level.

Table 8: Winning FNN architecture per level, selected by validation MAE in count space over a grid of four layer configurations, three learning rates, and two dropout rates (24 configurations per level).

Level	Layers	Learning rate	Dropout
community / hourly	256, 128, 64, 32	0.001	0.0
community / daily	128, 64, 32, 16	0.01	0.0
h3_7 / hourly	128, 64, 32, 16	0.0001	0.0
h3_7 / daily	128, 64, 32, 16	0.01	0.2
census / hourly	64, 32, 16	0.001	0.0
census / daily	64, 32, 16	0.001	0.2

Table 9: FNN test performance per level against the historical-mean baseline (skill = 1 − MAE_model / MAE_baseline), sorted by skill (MAE).

Level	MAE	RMSE	R^2	Skill (MAE)	Skill (RMSE)
community / hourly	2.620	9.678	0.925	+9.7%	+12.7%
h3_7 / hourly	1.041	7.358	0.883	+2.5%	+1.0%
census / hourly	0.276	2.218	0.844	−1.0%	−6.1%
community / daily	54.518	190.617	0.921	−10.0%	−4.8%
census / daily	6.355	42.094	0.819	−29.2%	−42.7%
h3_7 / daily	28.209	155.722	0.844	−31.9%	−28.7%

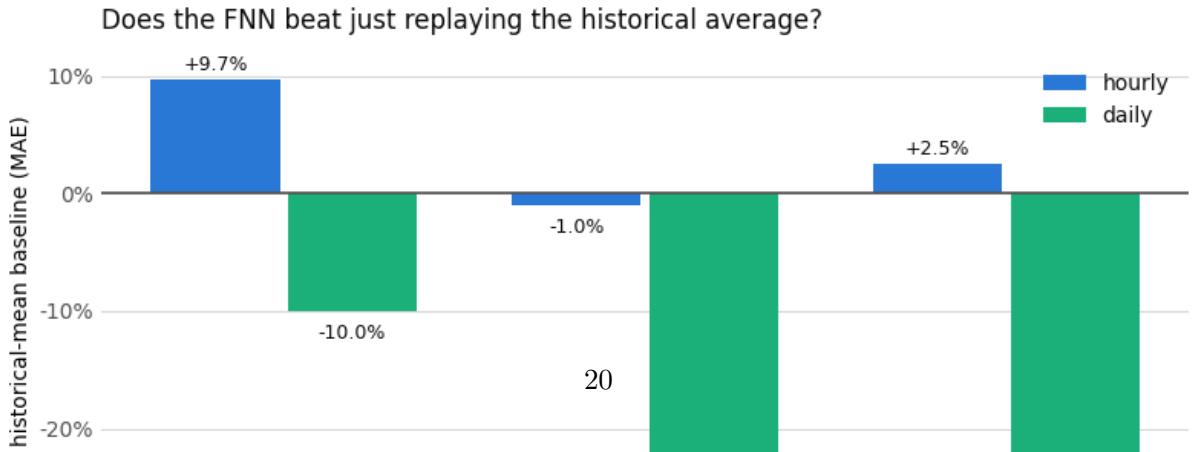
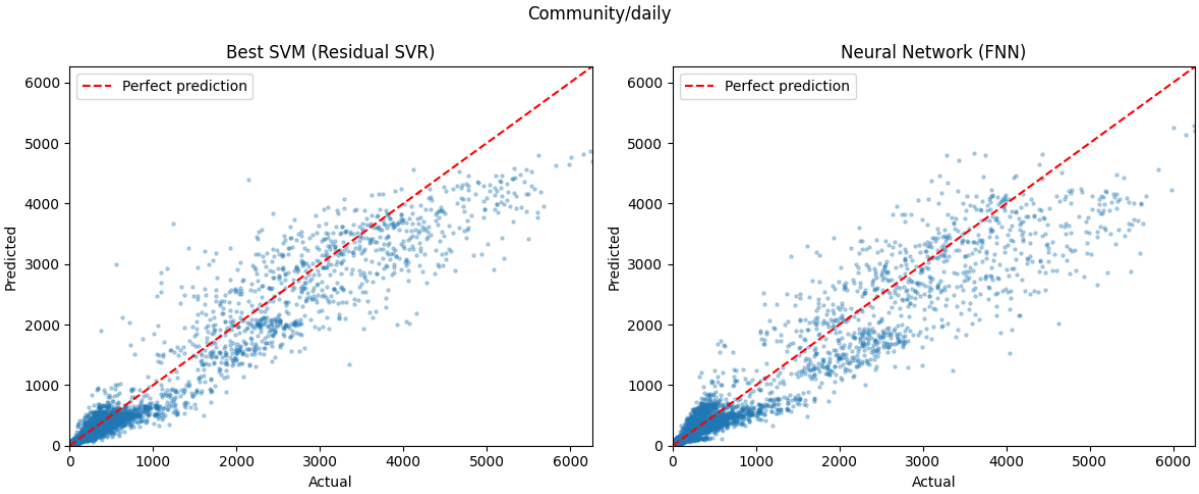


Table 10: Overall TEST performance at Community/daily and Community/hourly.

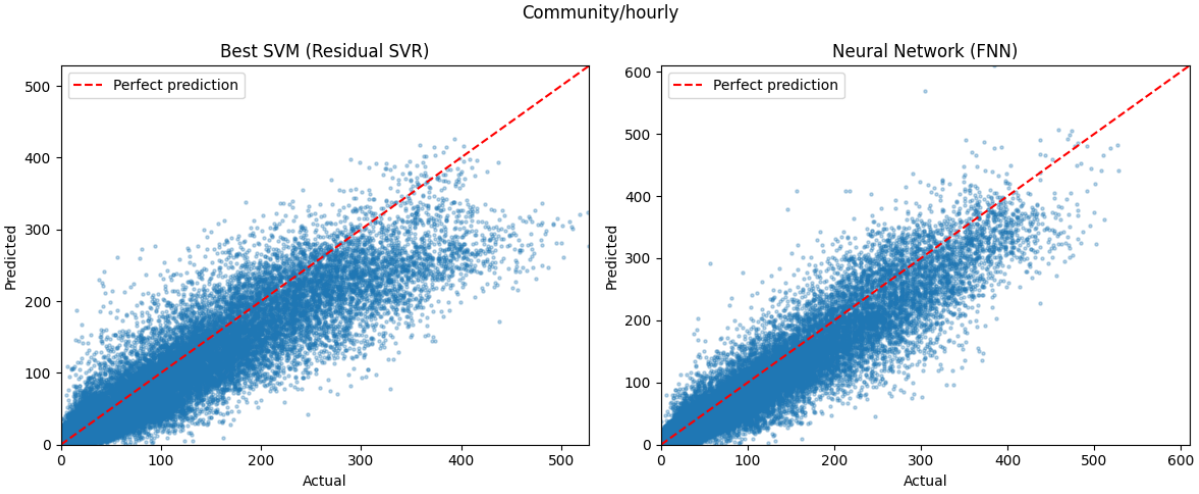
Model	MAE	RMSE	R2
Baseline	49.5750	181.9158	0.9283
Best SVM (Residual SVR)	46.3370	164.6925	0.9413
Neural Network (FNN)	54.5184	190.6171	0.9213

Table 11: Overall TEST performance at Community/daily and Community/hourly.

Model	MAE	RMSE	R2
Baseline	2.9029	11.0871	0.9019
Best SVM (Residual SVR)	2.8777	11.2611	0.8988
Neural Network (FNN)	2.6201	9.6776	0.9252



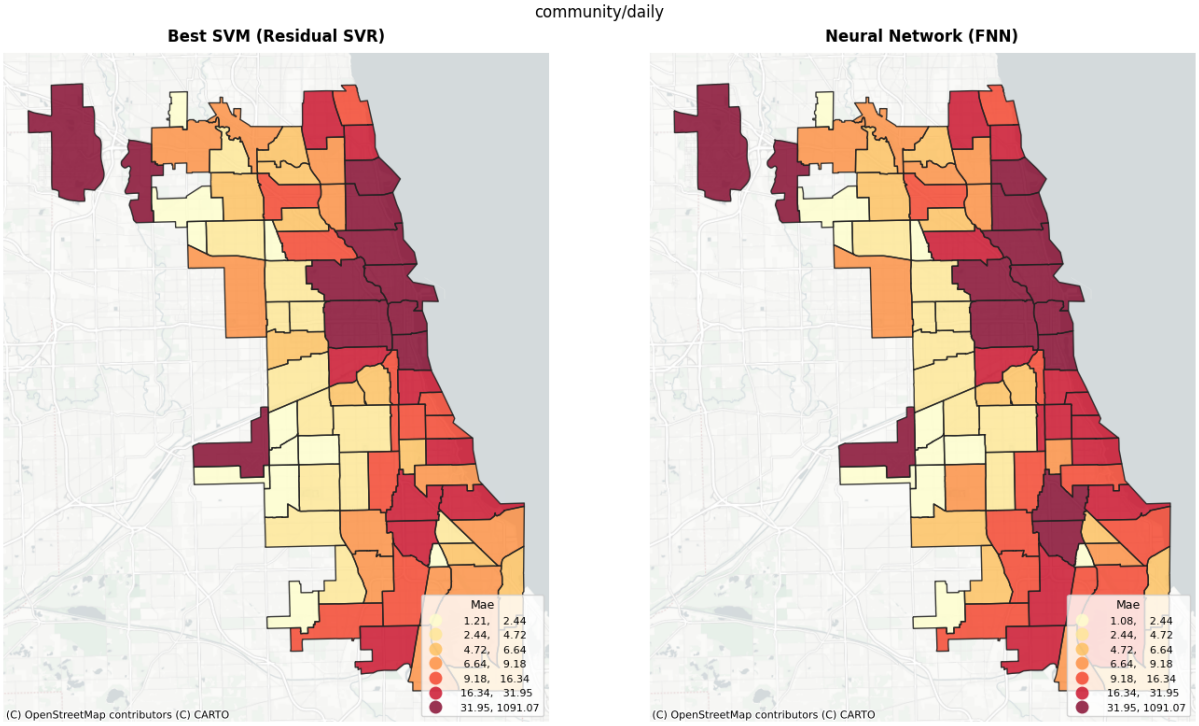
(a) Community/daily



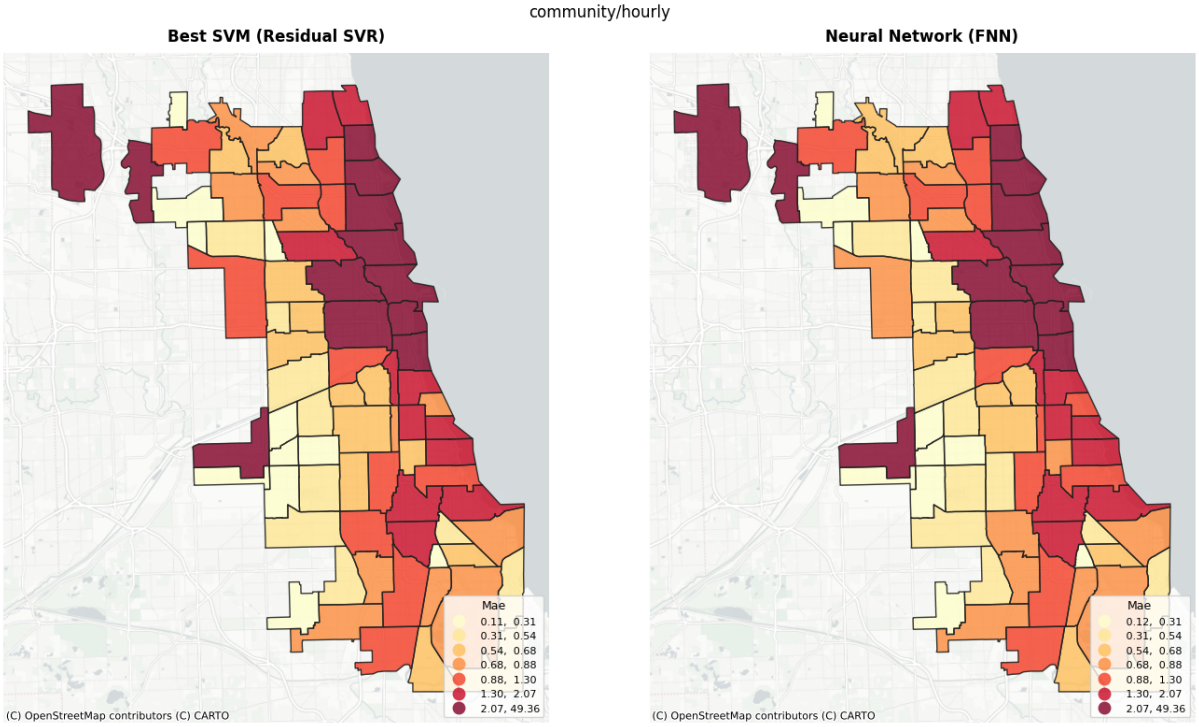
(b) Community/hourly

Figure 11: Actual vs. predicted demand per temporal resolution.

```
--- Community/daily ---
```

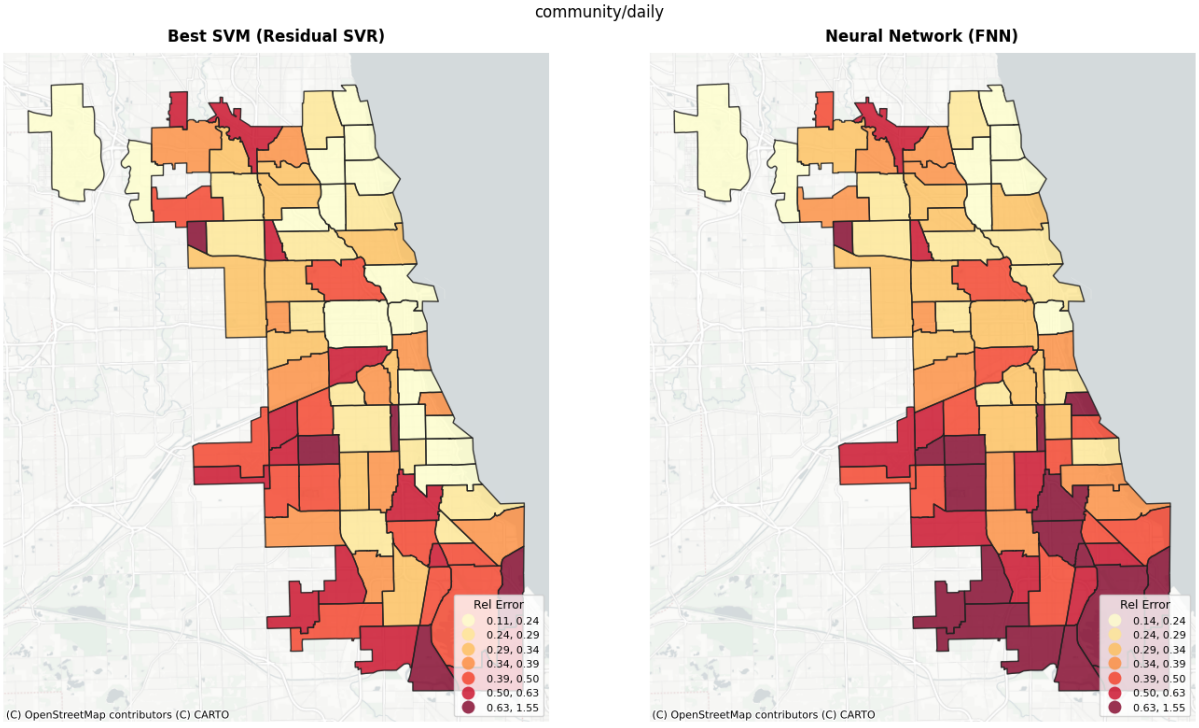


(a) MAE per community area per temporal resolution.

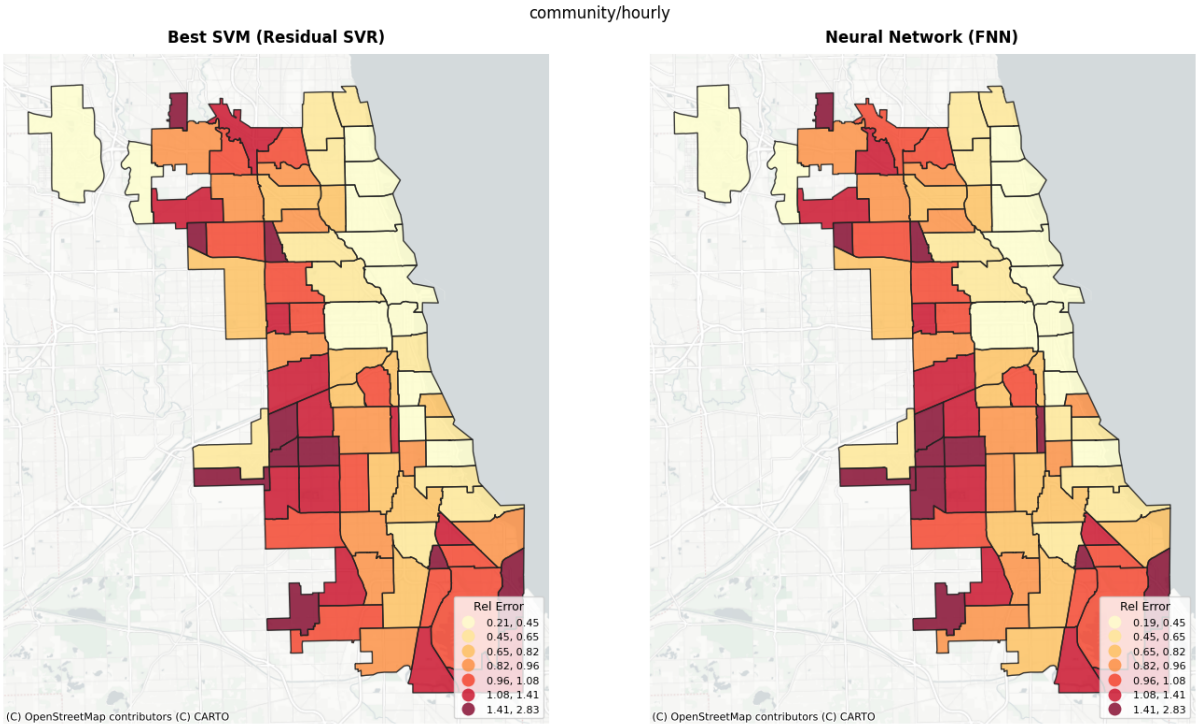


(b)

Figure 12

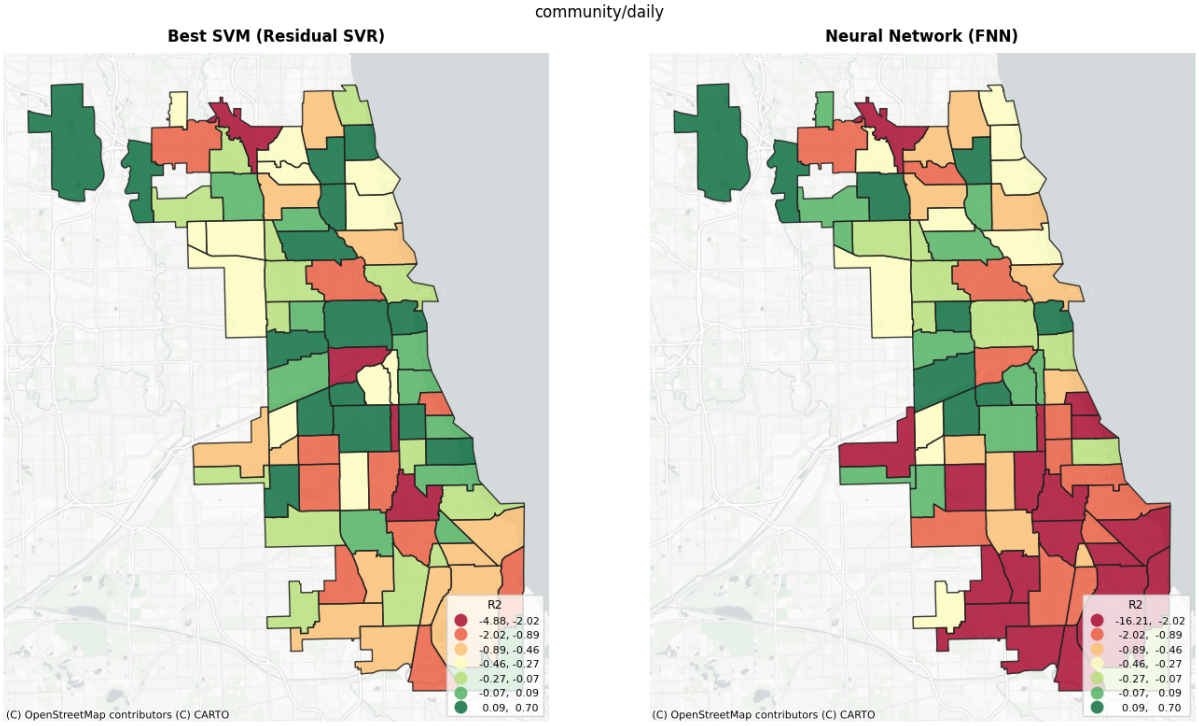


(a) Relative error per community area per temporal resolution.

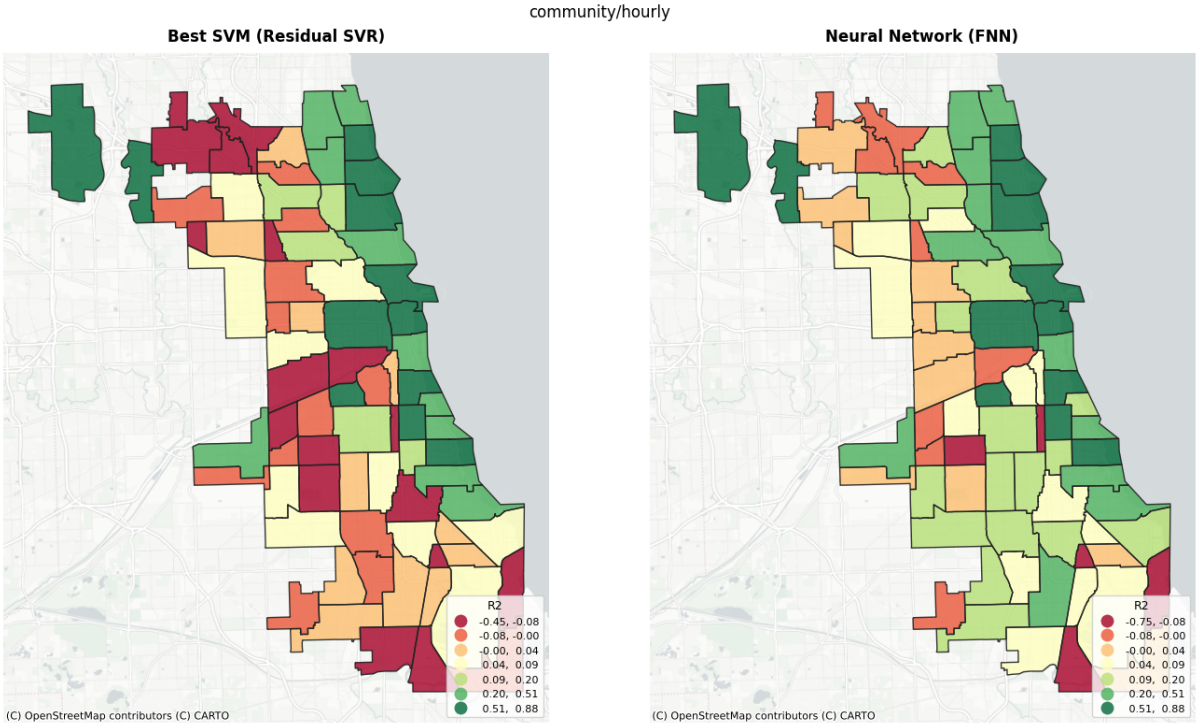


(b)

Figure 13

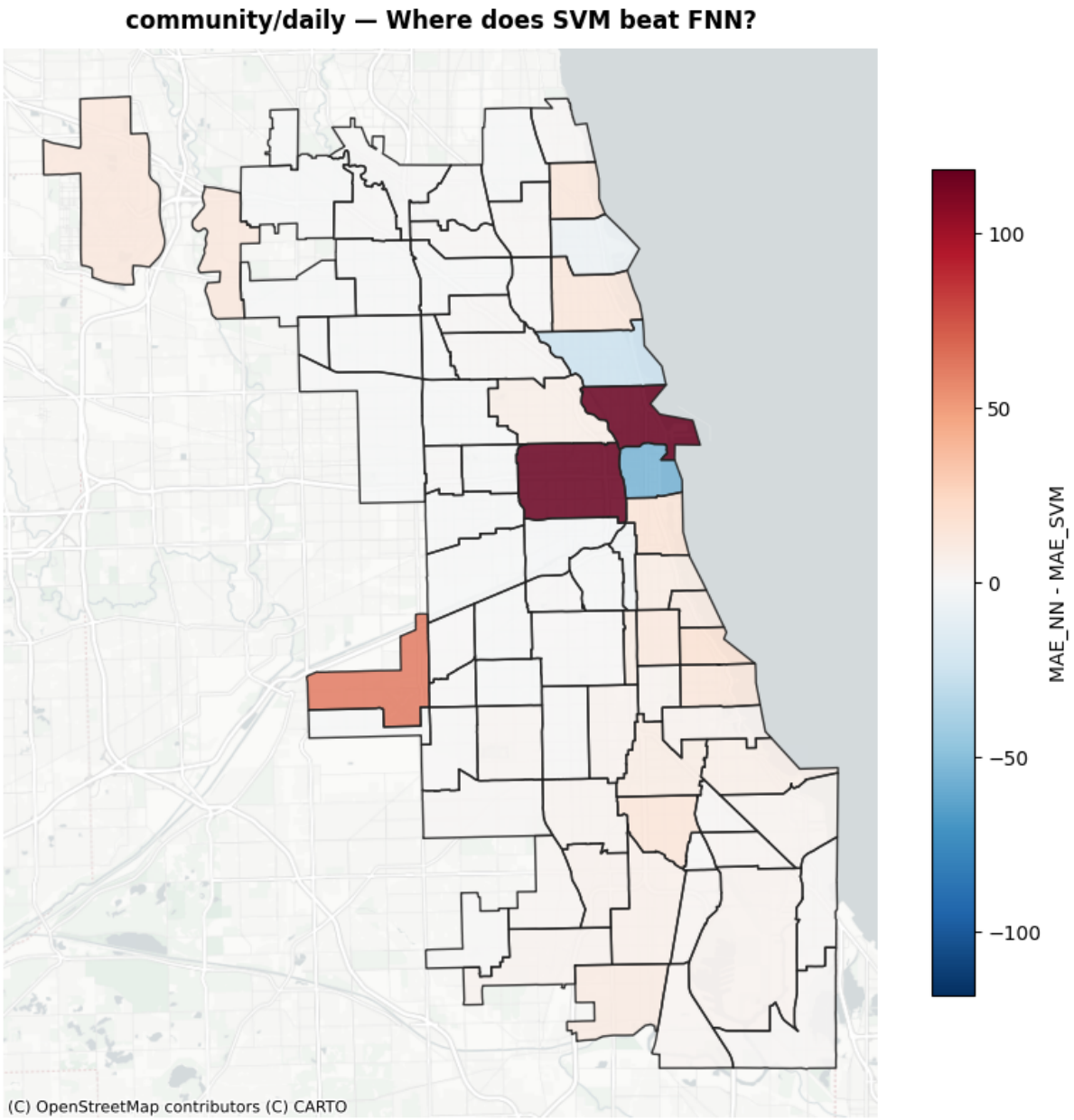


(a) R^2 per community area per temporal resolution.

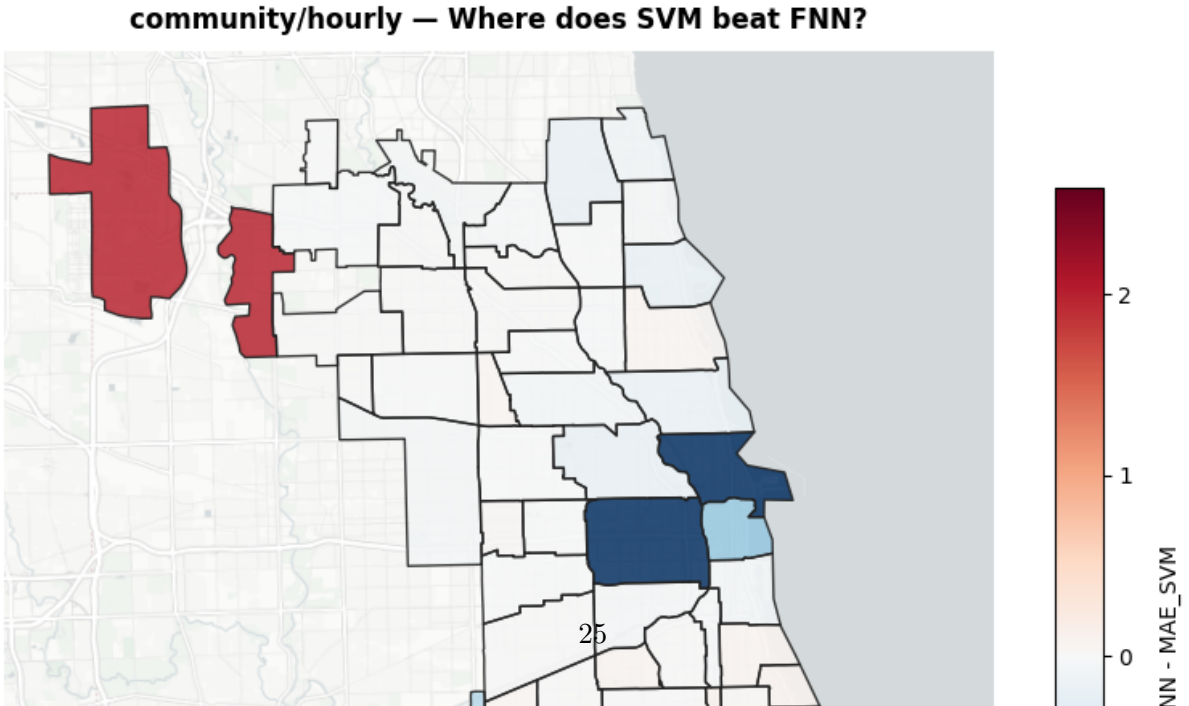


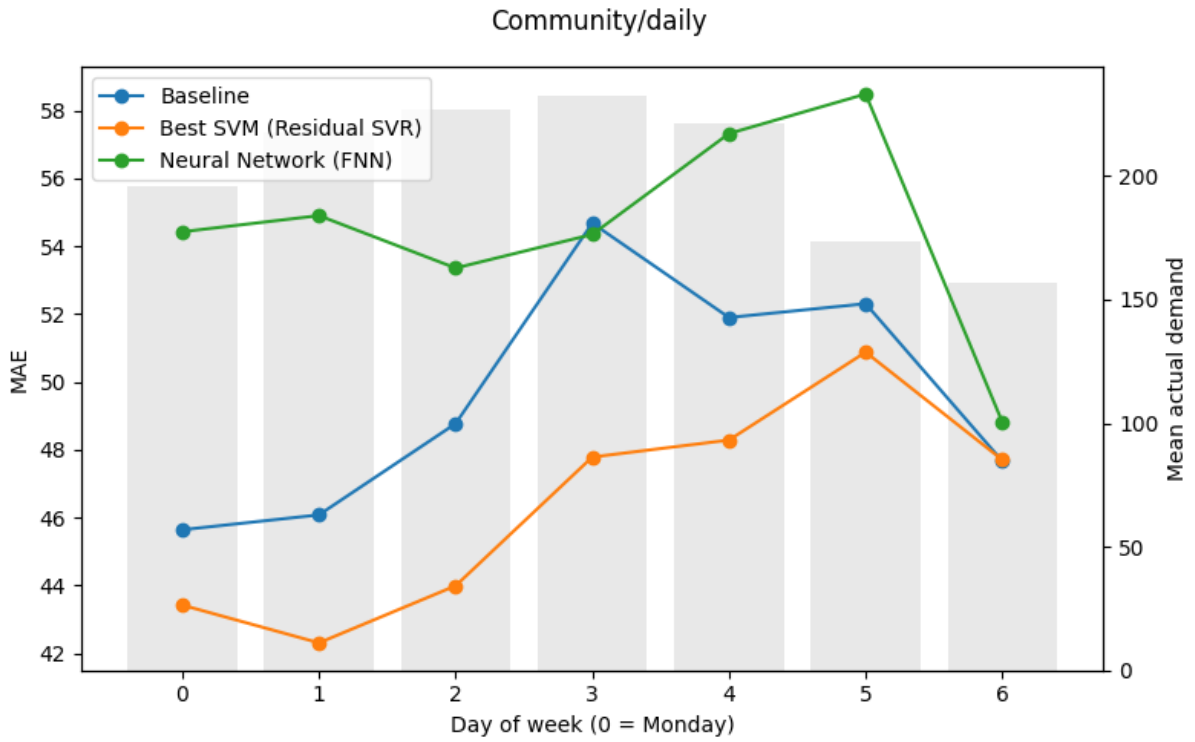
(b)

Figure 14

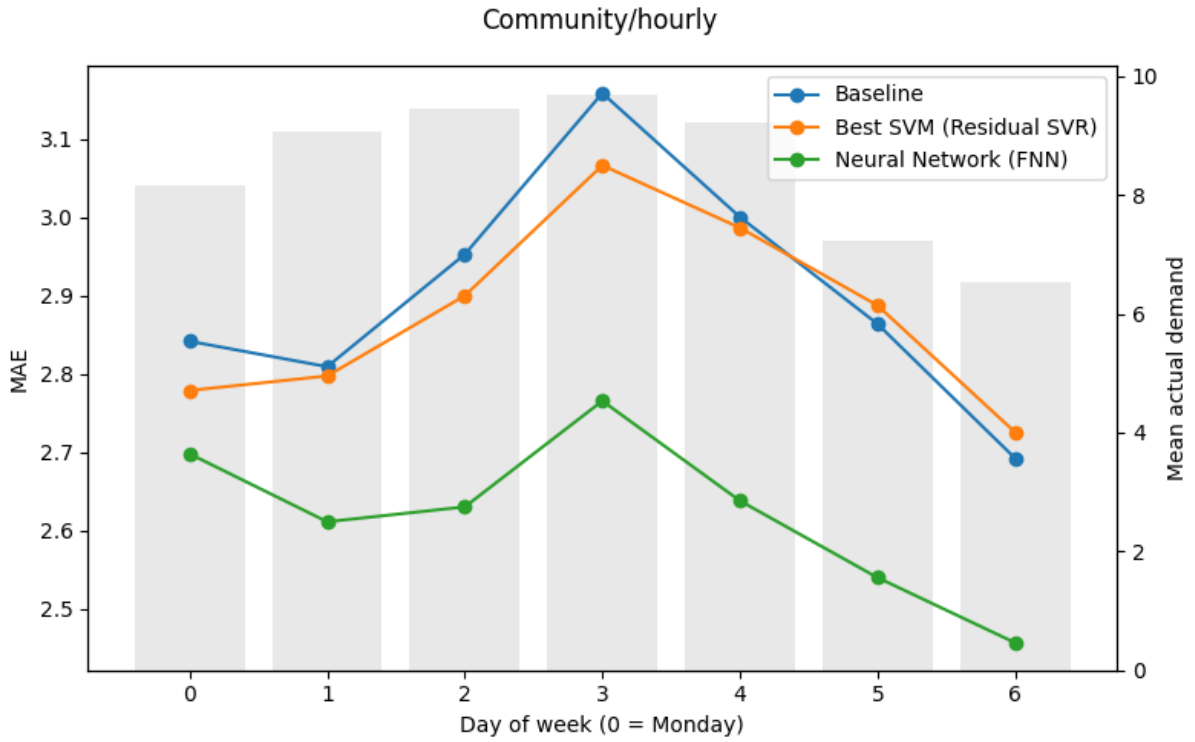


(a) MAE_NN - MAE_SVM per community area per temporal resolution. Blue = SVM wins, red = FNN wins.



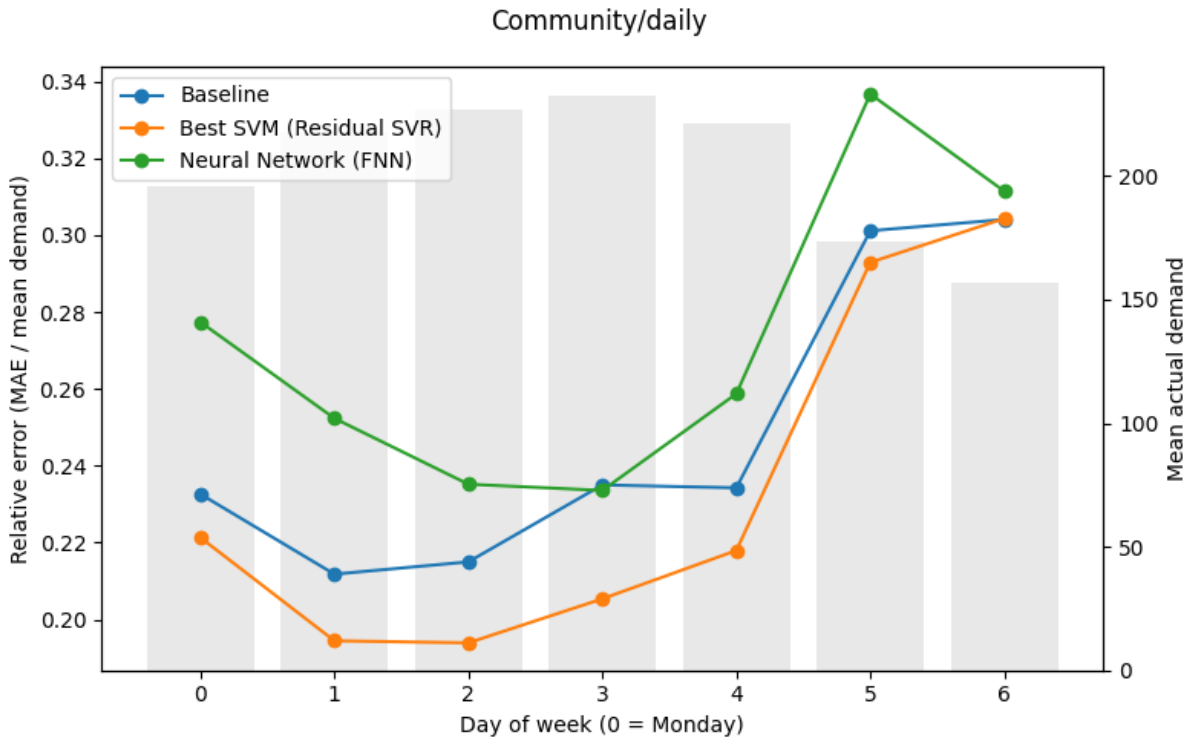


(a) Community/daily

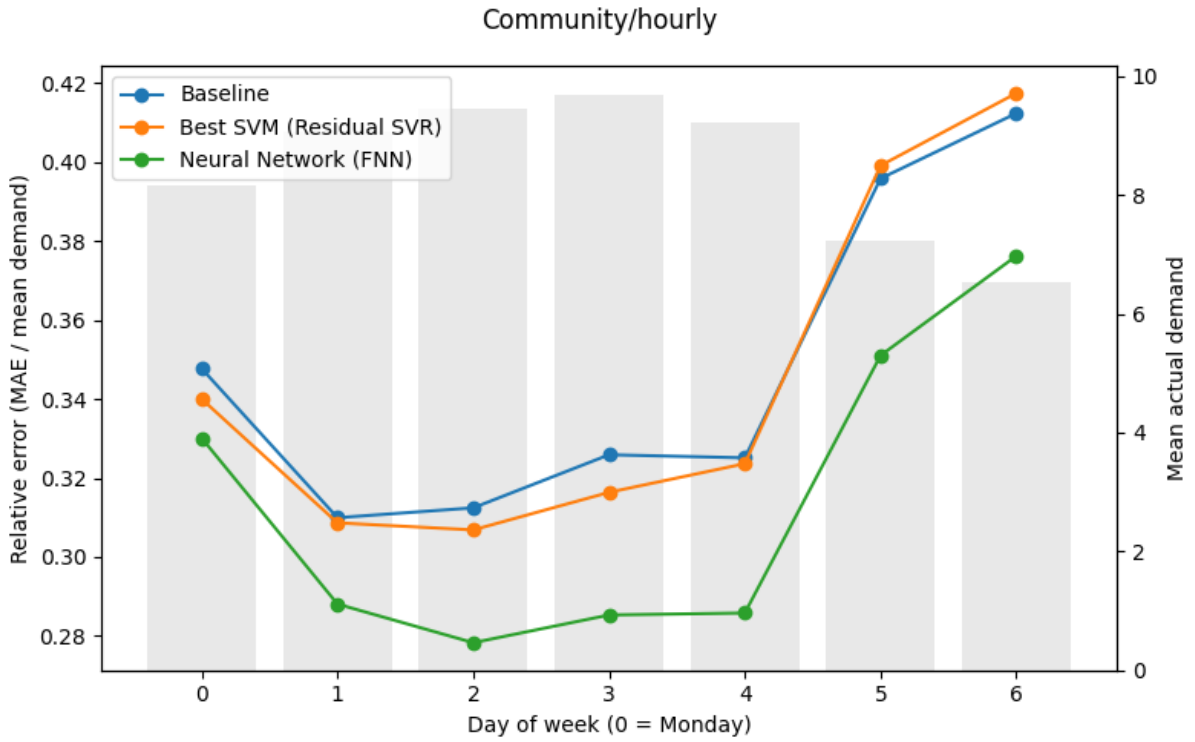


(b) Community/hourly

Figure 16: Test MAE by day of week per temporal resolution.



(a) community/daily



(b) community/hourly

Figure 17: Test relative error (MAE / mean demand) by day of week per temporal resolution.

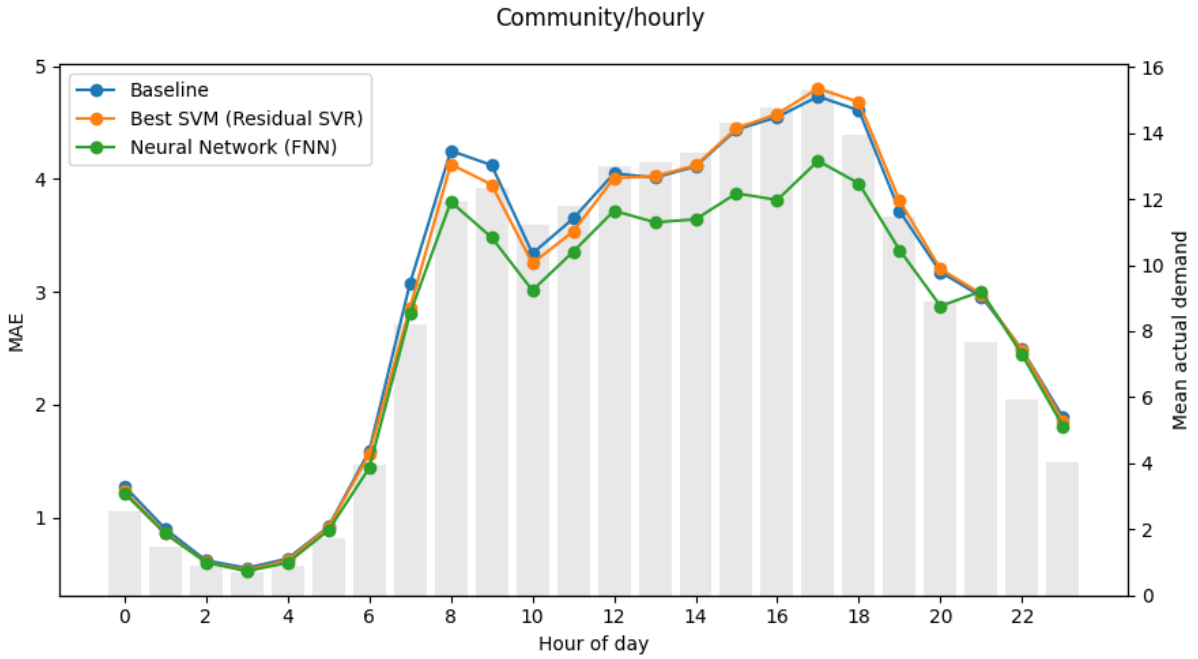


Figure 18: Test MAE by hour of day (Community/hourly).

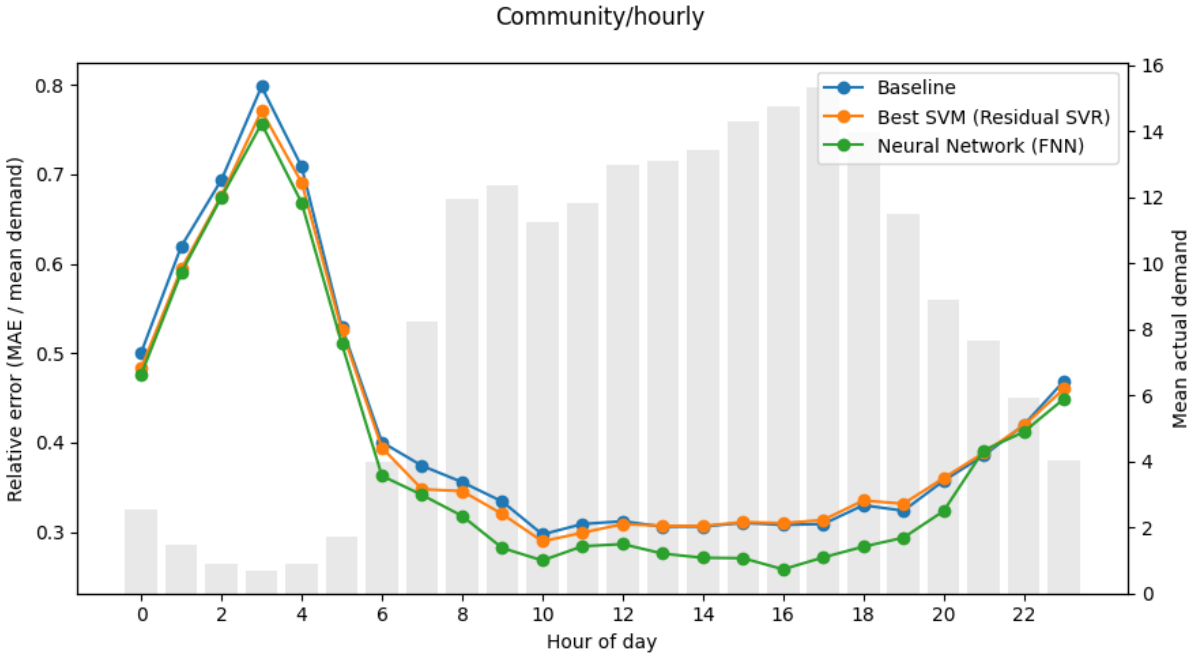
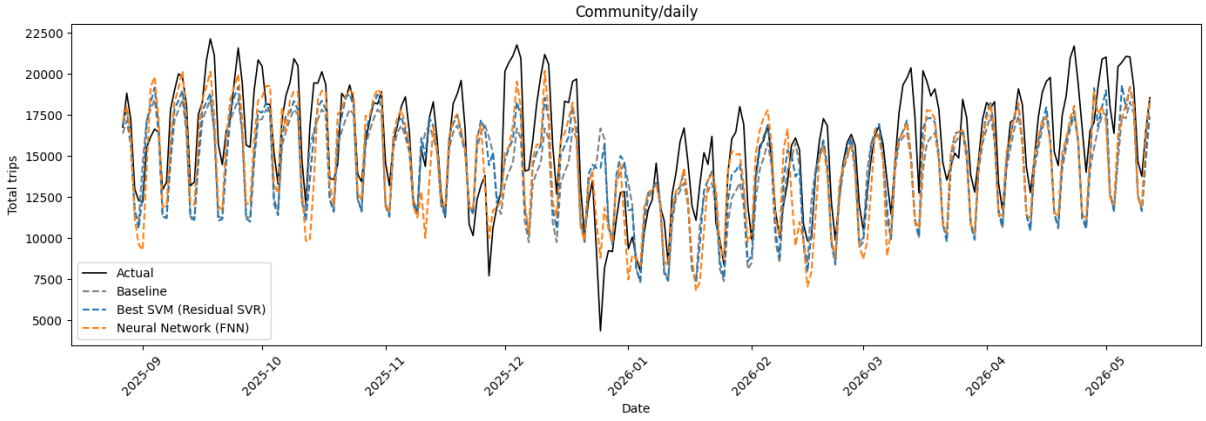
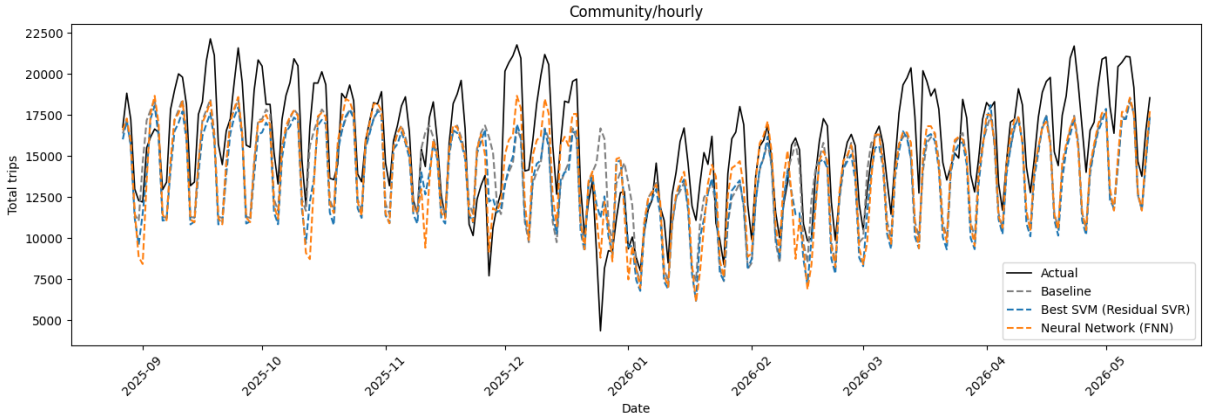


Figure 19: Test relative error (MAE / mean demand) by hour of day (Community/hourly).



(a) Community/daily



(b) Community/hourly

Figure 20: City-wide demand over the test period per temporal resolution.

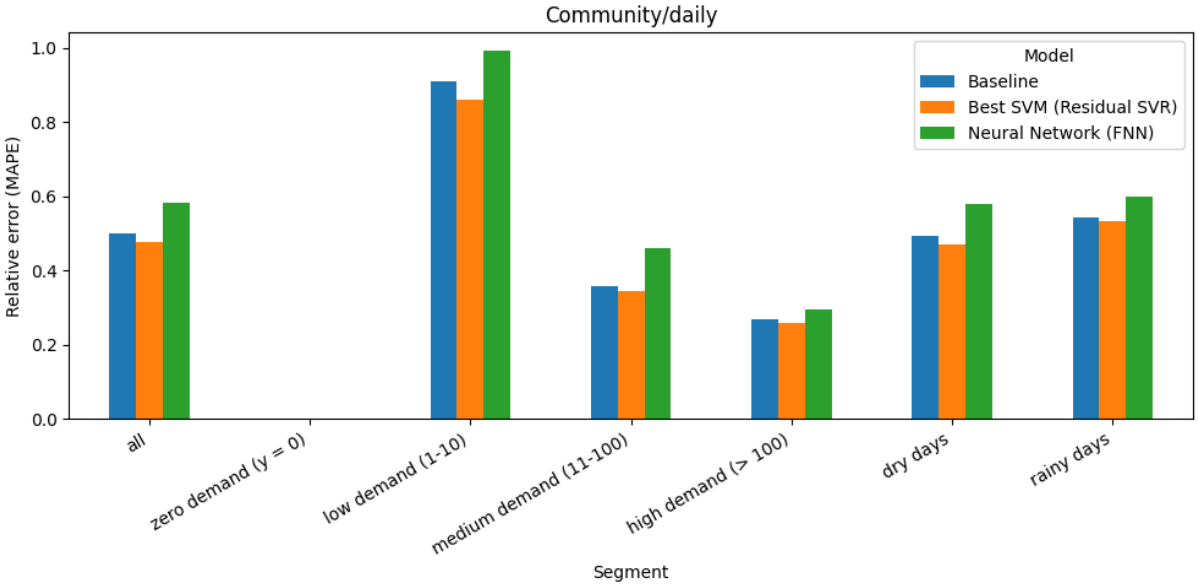
```
--- Community/hourly ---
```

Table 12: Test MAE, relative error, and skill score by demand band and weather per temporal resolution.

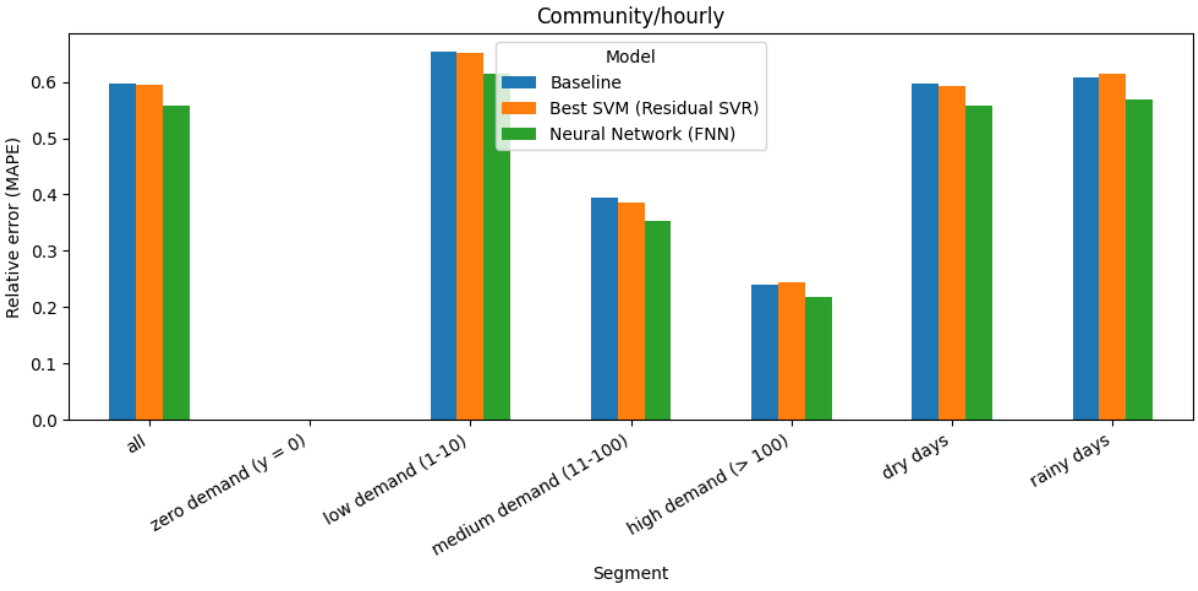
segment	Model	n	MAE	rel_error	skill_score
all	Baseline	19943	49.575	0.499	0.000
	Best SVM (Residual SVR)	19943	46.337	0.476	0.065
	Neural Network (FNN)	19943	54.518	0.582	-0.100
zero demand (y = 0)	Baseline	250	2.873	NaN	0.000
	Best SVM (Residual SVR)	250	2.721	NaN	0.053
	Neural Network (FNN)	250	2.893	NaN	-0.007
low demand (1-10)	Baseline	5557	3.790	0.911	0.000
	Best SVM (Residual SVR)	5557	3.598	0.862	0.051
	Neural Network (FNN)	5557	4.391	0.992	-0.158
medium demand (11-100)	Baseline	10788	10.166	0.359	0.000
	Best SVM (Residual SVR)	10788	9.819	0.345	0.034
	Neural Network (FNN)	10788	13.234	0.460	-0.302
high demand (> 100)	Baseline	3348	256.040	0.267	0.000
	Best SVM (Residual SVR)	3348	238.203	0.260	0.070
	Neural Network (FNN)	3348	274.604	0.294	-0.073
dry days	Baseline	17633	49.678	0.493	0.000
	Best SVM (Residual SVR)	17633	46.481	0.469	0.064
	Neural Network (FNN)	17633	54.490	0.579	-0.097
rainy days	Baseline	2310	48.789	0.544	0.000
	Best SVM (Residual SVR)	2310	45.238	0.533	0.073
	Neural Network (FNN)	2310	54.735	0.601	-0.122

Table 13: Test MAE, relative error, and skill score by demand band and weather per temporal resolution.

segment	Model	n	MAE	rel_error	skill_score
all	Baseline	478632	2.903	0.597	0.000
	Best SVM (Residual SVR)	478632	2.878	0.595	0.009
	Neural Network (FNN)	478632	2.620	0.559	0.097
zero demand (y = 0)	Baseline	230578	0.507	NaN	0.000
	Best SVM (Residual SVR)	230578	0.409	NaN	0.194
	Neural Network (FNN)	230578	0.428	NaN	0.156
low demand (1-10)	Baseline	202066	1.379	0.653	0.000
	Best SVM (Residual SVR)	202066	1.389	0.651	-0.007
	Neural Network (FNN)	202066	1.349	0.613	0.022
medium demand (11-100)	Baseline	33792	12.172	0.394	0.000
	Best SVM (Residual SVR)	33792	11.927	0.387	0.020
	Neural Network (FNN)	33792	10.801	0.353	0.113
high demand (> 100)	Baseline	12196	47.763	0.240	0.000
	Best SVM (Residual SVR)	12196	49.147	0.243	-0.029
	Neural Network (FNN)	12196	42.466	0.219	0.111
dry days	Baseline	421036	2.868	0.596	0.000
	Best SVM (Residual SVR)	421036	2.849	0.592	0.007
	Neural Network (FNN)	421036	2.590	0.557	0.097
rainy days	Baseline	57596	3.157	0.607	0.000
	Best SVM (Residual SVR)	57596	3.084	0.614	0.023
	Neural Network (FNN)	57596	2.837	0.569	0.101



(a) community/daily



(b) community/hourly

Figure 21: Test relative error (MAE / mean demand) by segment per temporal resolution.

A.14 Reinforcement Learning (4.1)

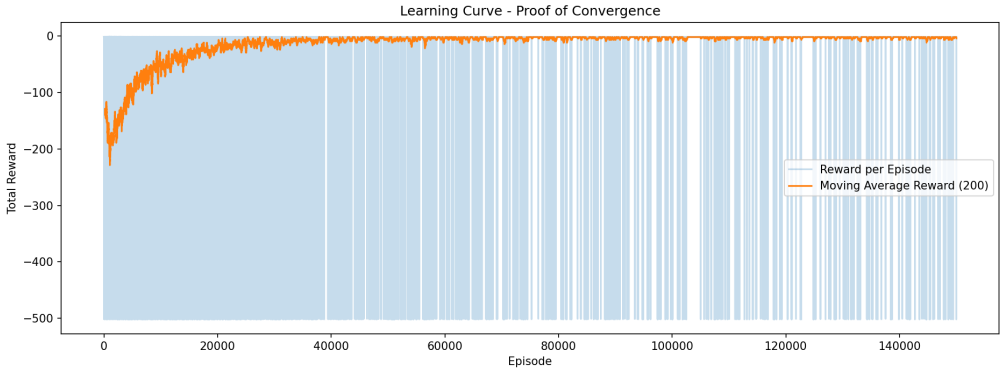


Figure 22: Figure 1a: Training score over time.

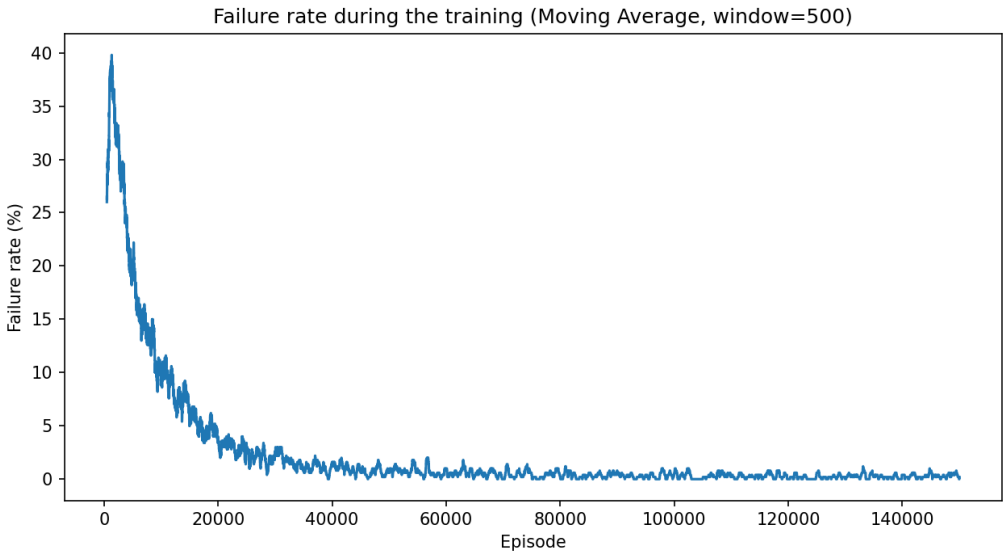


Figure 23: Figure 1b. Probability of running out of energy over time.

```
--- Calculating Naive Strategy Baseline (2000 days) ---
Naive Success Rate: 100.00%
Average Naive Charging Cost per day: $1.92
```

Table 14: Table Naive vs. Tabular Q-Learning.

	Strategy	Avg. Daily Cost	Probability of Running Out
0	Naive (always full power)	$1.92	~0.14%
1	Tabular Q-Learning	$1.50 ± $0.02	0.11% ± 0.09%

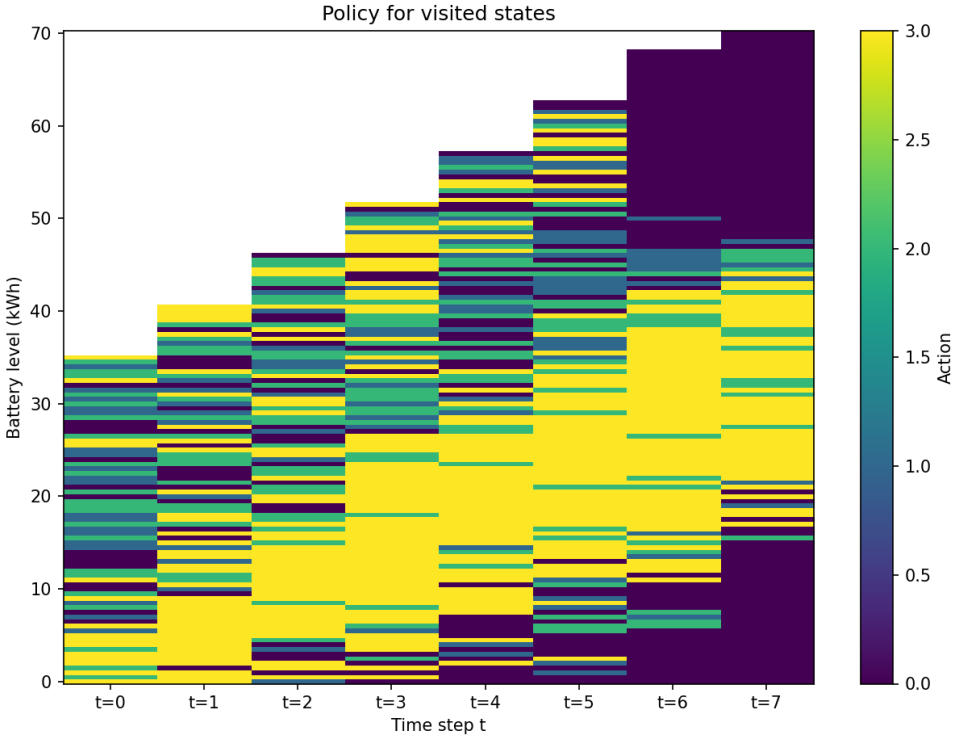


Figure 24: Figure 2. Learned charging policy. The color indicates the chosen charging speed for each battery-level/time combination.

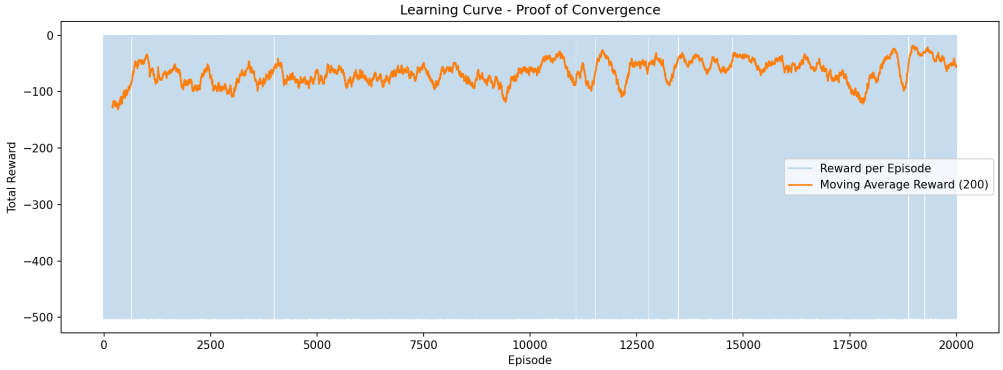


Figure 25: Figure 3a. DQN training score.

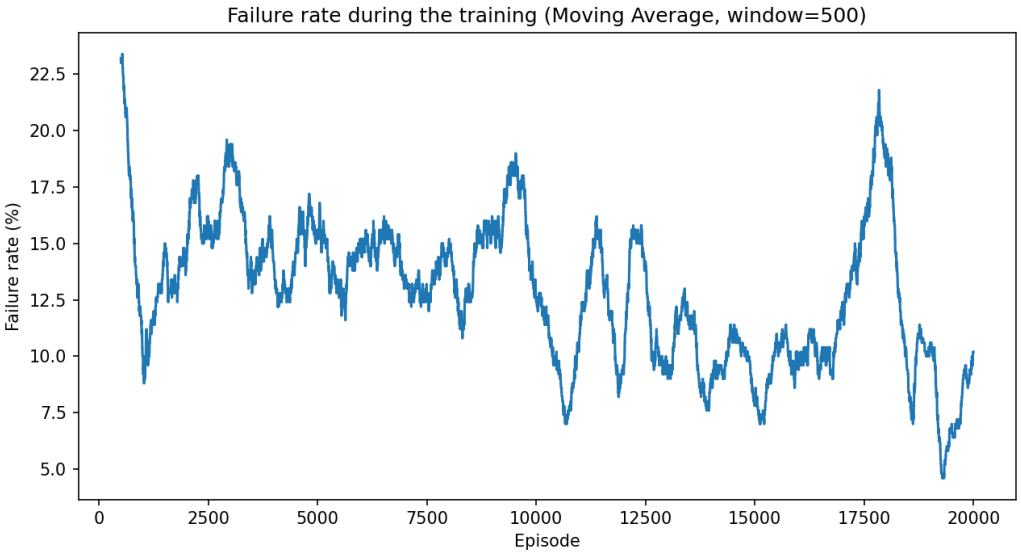


Figure 26: Figure 3b. DQN failure rate during training.

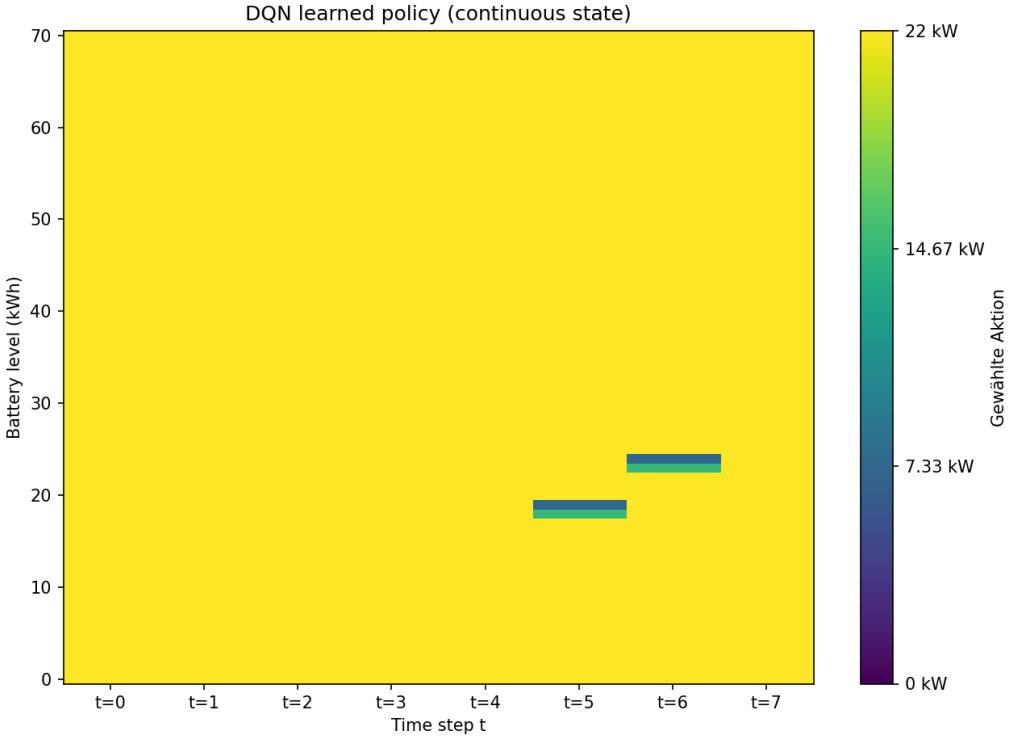


Figure 27: Figure 4. Best DQN policy found across the five training runs. The Q-table stores entries only for states that were actually visited during training (visit_counts > 0) and thus, combinations of time step and battery level that were never reached remain empty. The DQN network, by contrast, is a continuous function that produces an output for any given input. This includes states that never occurred during training (e.g. battery=60 at t=0). Thereby, both plots differ in terms of their appearance.

Table 15: Table DQN Summary.

	Strategy	Avg. Daily Cost	Probability of Running Out
0	Deep Q-Network (DQN)	\$1.46 ± \$0.24	1.16% ± 0.95%

Table 16: Table DQN Per-Seed Breakdown.

	Run (seed)	Avg. Daily Cost	Probability of Running Out	Best checkpoint found at episode
0	0	\$1.29	2.82%	16,000
1	1	\$1.92	0.00%	8,000
2	2	\$1.28	1.48%	18,000
3	3	\$1.53	0.70%	12,000
4	4	\$1.31	0.78%	12,000

No additional labeled figures or tables from this notebook.

References

Yan, J., Xiang, L., Wu, C., & Wu, H. (2020). City-scale taxi demand prediction using multisource urban geospatial data. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, *43*, 213–220.